Mastering Identus: A Developer Handbook

Jon Bauer, Roberto Carvajal

2024-04-05

Table of contents

Welcome

Welcome to the book!

Copyright

License

Book Content:

The content of the book (text, printed code, and images) are licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0

The intent here is to allow anyone to use or excerpt from the book as long as they credit the source. In general we are very open to having our work shared non-commercially. We are copyrighting this book to protect the ability to version and revise the work officially, and to keep open the door for a print version if the opportunity arises and/or makes sense.

If you'd like to commercially reproduce contents from this book, please contact us.

Applications:

The example code that accompanies this book is licensed under a MIT License

This allows anyone to freely take, modify, or redistribute the code as long as they also assign the MIT license to it.

Dedication

Preface

Explains who the authors are and why we were motivated to write this book.

Section I

Introduction

This is a developer-centric book about creating and launching Self-Sovereign applications with Identus. We aim to show the reader how to configure, build and deploy a complex idea from scratch.

This is not a book about Self-Sovereign Identity. There are already great resources available on that topic. If you're new to the idea of SSI or Identus, we recommend the excellent resources listed below as a pre-requisite to this text.

- Self-Sovereign Identity by Alex Preukschat, Drummond Reed, et al. This is the definitive book on SSI and it's ecosystem of topics.
- Identus Documentation

It should be noted that Identus is still new and at the time of writing this book, there are very few best practices, in fact, very little practices at all. A handful of adventurous developers have been building on the platform and sharing their experiences, and we hope to share our own learnings in hopes of magnifying that knowledge and helping developers skip the common pitfalls and bring their ideas to market.

We hope this text will be accessible to anyone who is curious about building decentralized digital identity applications.

We hope that newcomers will be able to use this text to skip common pitfalls and misunderstandings, and bring their ideas to market faster.

We're glad you could join us :)

SSI Basics

What SSI Means in This Book

Self-Sovereign Identity (SSI) describes identity systems built around user-held credentials and verifiable proofs, rather than a single identity provider that stores the user's account profile. A person, organization, device, or software agent can receive a credential from an issuer, store it in a wallet or agent, and present proof to a verifier. The verifier checks the proof and applies its own policy without a custom account integration with every issuer.

SSI does not make holder-provided claims trustworthy by itself. A verifier checks who issued the credential, what the credential says, whether the proof is valid, whether the credential is still in good standing, and whether the issuer is acceptable for that transaction. SSI changes custody and exchange of identity data; governance, law, contracts, and reputation still shape trust decisions.

This chapter uses standard terms from the W3C Decentralized Identifiers (DIDs) v1.0 Recommendation and the W3C Verifiable Credentials Data Model v2.0 Recommendation. DID Core defines DIDs as identifiers for verifiable, decentralized digital identity that can be decoupled from centralized registries, identity providers, and certificate authorities. The VC Data Model defines credentials, presentations, issuers, holders, subjects, and verifiers. The Identus Quick Start Guide describes Identus as core libraries for SSI interactions between issuers, holders, and verifiers.

The SSI Interaction

A typical exchange has four steps.

1. An issuer makes claims about a subject and packages those claims as a credential.
2. A holder receives the credential and stores it in a wallet or agent.
3. A verifier asks the holder for proof that satisfies a specific request.
4. The holder returns a presentation, and the verifier checks cryptography, status, and business policy.

Example: a university issues a degree credential to Alice. Alice receives the credential in her wallet and acts as holder. The degree describes Alice, making Alice the subject. An employer acts as verifier and requests proof that Alice has a computer science degree from an accredited university. Alice's wallet creates a presentation from the credential. The employer's software checks that the university signed the credential, that Alice can prove control of the holder keys required by the credential format, that the issuer has not suspended or revoked the credential, and that the university is accepted under the employer's hiring policy.

Policy evaluation is validation. Cryptographic verification can show that an issuer's key secured the credential and that no one modified the credential after issuance. It cannot show that the issuer is acceptable for a particular decision. A diploma from an unknown training provider can pass cryptographic verification and still fail an employer's policy. The W3C VC specification describes validation as the verifier's business-rule check for whether a credential is proper for a particular use.

DIDs and DID Documents

A Decentralized Identifier (DID) is a URI. It has a `did:` scheme, a DID method, and a method-specific identifier. In `did:prism:4a9bce8d72e4c30017c42f2b`, the method is `prism`; the last segment is the identifier data defined by that method.

The DID method tells software how to create, resolve, update, and deactivate that class of DID. DID Core does not require one storage technology. A DID method can use a distributed ledger, a database, a peer-to-peer network, or another verifiable data registry, as long as the method defines the required operations.

A DID resolves to a DID Document. DID Documents express public verification material, verification relationships, and services. A DID Document can tell software which public keys can be used for authentication, assertion, key agreement, capability invocation, or capability delegation. It can list service endpoints for communication protocols such as DIDComm. It should not contain secrets, private keys, or a personal profile.

The DID subject and DID controller may be the same entity, but they do not have to be. The subject is the person, organization, device, data model, or other thing identified by the DID. The controller is the entity that can change the DID Document according to the DID method. A company DID might identify the company as subject, and the company might delegate key operations to a Cloud Agent it controls.

DIDs can create correlation risk. DID Core warns that globally unambiguous identifiers can link activity across contexts. Pairwise DIDs reduce that risk by giving each relationship its own pseudonymous DID. Apply the same privacy review to DID Documents: reused keys, unusual service endpoints, or descriptive properties can still link activity across relationships.

Verifiable Credentials

A credential is a set of claims made by an issuer. A Verifiable Credential (VC) is a credential with a securing mechanism that lets software detect tampering and check authorship. The VC can be about a person, organization, device, account, shipment, software artifact, or any other subject.

The issuer decides what to claim and signs or otherwise secures the credential. The holder stores the credential. The subject is the entity the claims describe. The holder and subject often match, but not always. A parent can hold a credential about a child. A company officer can hold a credential about the company. A device fleet manager can hold credentials about devices.

The data model is separate from the credential format. W3C VC 2.0 defines the model. Implementations can secure and transport credentials through different formats and protocols. Identus chapters later in the book use JWT-VC, SD-JWT-VC, and AnonCreds in implementation flows, so the right choice depends on the credential formats supported by the agents and wallets in that flow.

Credential status is part of production verification. W3C VC 2.0 defines `credentialStatus` for discovering status information such as suspension or revocation. The details depend on the status method. A verifier that accepts a credential without checking status may accept a credential the issuer no longer stands behind.

Verifiable Presentations

A Verifiable Presentation (VP) is data derived from one or more credentials and shared with a specific verifier. A presentation can include one credential, parts of one credential, or data from more than one credential, depending on the credential format and proof mechanism.

A holder uses presentations to respond to verifier requests. A verifier asks for the data it needs. The wallet shows the request to the holder. The holder approves or rejects the disclosure. The wallet then builds a presentation that fits the request and the credential format.

Selective disclosure depends on the credential format and proof mechanism. The W3C VC Data Model defines selective disclosure as the holder's ability to make fine-grained choices about what to share. The same specification treats data minimization as a best practice: verifiers should request the minimum information needed for a transaction. A credential format that supports selective disclosure can let Alice prove she is over 21 without revealing her full birth date. A format that lacks that feature may require sharing more data.

The Roles in the Trust Triangle

SSI discussions often use the "Triangle of Trust" to describe issuer, holder, and verifier. Treat the triangle as roles in an interaction, not permanent labels for people or organizations.

An issuer creates a credential and takes responsibility for the claims it makes. A department of motor vehicles can issue a driver's license credential. A university can issue a degree credential. A gym can issue a membership credential. The issuer's authority comes from the surrounding context: law, accreditation, contract, community rules, reputation, or governance.

A holder receives credentials and creates presentations from them. The holder may be a person using a mobile wallet, a company using a Cloud Agent, or software managing credentials for devices. Holder control means the holder decides whether to present a credential in a given interaction, subject to wallet design, custody model, law, and policy.

A verifier receives a presentation and decides whether to accept it. Verification usually checks syntax, cryptographic proofs, issuer keys, holder binding, credential status, and schema. Validation checks business rules. For a bar, "over 21 and issued by a recognized state DMV" may be enough. For a hospital employee credential, the verifier may need role, facility, license status, and trust registry checks.

Roles are interaction-specific. The same entity can hold one credential, verify another credential, and issue a third credential.

Trust Registries and Governance

A DID can prove control over keys. A credential can prove that an issuer made a claim. A verifier still needs a way to decide whether that issuer has the right authority for the verifier's context. Trust registries publish that authorization information for a defined ecosystem.

The Trust over IP Foundation describes a trust registry query in plain terms: "Does entity X hold authorization Y under ecosystem governance framework Z?" The ToIP Trust Registry Query Protocol v2.0 announcement frames TRQP as a read-only way to discover who is authorized to do what within a digital trust ecosystem. LF Decentralized Trust describes a trust registry as a system of record containing authoritative information that relying parties use for trust decisions.

A trust registry does not create authority by itself. Authority comes from the governance framework behind it. For a real estate credential, the registry might list licensed agents and the credential types they can issue or verify. For a university degree, a registry might list accredited institutions. For a supply-chain credential, it might list auditors accepted by a particular industry group.

A verifier should treat technical verification and acceptance policy as separate checks. Verification checks proofs, keys, syntax, and status. Acceptance policy checks whether the issuer and credential type fit the relying party's rules.

Exchange Protocols

DIDs and VCs define data models and verification material. Applications still need protocols to exchange messages, issue credentials, and request presentations.

DIDComm Messaging v2.1 defines encrypted message formats used by many agent-to-agent SSI systems. DIDComm encrypted messages hide content from everyone except authorized recipients, disclose and prove the sender to those recipients in authenticated mode, and provide integrity guarantees. Identus uses DIDComm V2 for Cloud Agent-to-Cloud Agent communication and for many agent flows.

The OpenID Foundation defines OAuth-based protocols for credential exchange. OpenID4VCI 1.0 defines an API for issuing Verifiable Credentials. OpenID4VP 1.0 defines a protocol for requesting and presenting credentials. Production ecosystems may support DIDComm, OpenID4VCI, OpenID4VP, QR-code handoff, offline presentation, or a mix of these paths.

How This Maps to Identus

Identus gives developers components for these SSI roles, so application code can use agents, SDKs, and APIs instead of reimplementing the standards stack.

The Identus Cloud Agent can issue, hold, and verify VCs; manage DIDs and DID-based connections; expose a REST API; and use DIDComm V2 for agent communication. Typical uses include custodial wallets, enterprise issuers, verifier services, and backend workflows.

The Identus DID management documentation explains that Cloud Agent manages PRISM DIDs and Peer DIDs. PRISM DIDs fit public issuer identity and ledger-backed resolution. Peer DIDs fit DIDComm activities where relationship-specific identifiers reduce correlation.

The Identus Quick Start Guide describes PRISM Node as the Verifiable Data Registry component used to anchor key information for issuance and verification, and describes mediators as services that store and relay messages between Cloud Agents and wallet SDKs. PRISM Node supports public DID resolution. Mediators handle message relay for offline or intermittently connected agents without reading encrypted DIDComm content.

Developer Takeaways

SSI lets issuers, holders, and verifiers exchange claims with cryptographic proof and holder participation. DIDs identify and resolve verification material. DID Documents publish public keys and services, not private identity profiles. Verifiable Credentials carry issuer claims. Verifiable Presentations let holders respond to verifier requests. Verification checks proofs and status. Validation applies policy. Trust registries and governance help verifiers decide which issuers and credential types count in a specific ecosystem.

In Identus, these ideas map to components: PRISM Node for public DID resolution, Cloud Agent for server-side issuer/holder/verifier work, Edge Agent SDKs for wallets, DIDComm for secure agent messages, and mediators for routing when edge agents are offline.

DIDs and DID Documents

Overview

A **Decentralized Identifier (DID)** is a URI for a subject. The subject can be a person, organization, device, software agent, data model, or another entity chosen by the DID controller. The W3C DID Core specification defines a DID as a URI with three parts: the `did:` scheme, a method name, and a method-specific identifier.

The DID itself is only the identifier. A resolver resolves the DID and returns a DID resolution result. A successful result includes a **DID Document**, resolution metadata, and DID Document metadata. The W3C DID Resolution specification defines this resolver interface and leaves the method-specific retrieval steps to the DID method.

A **DID Document** publishes the public information software needs to interact with the DID subject: verification methods, verification relationships, and services. A DID Document should not contain private keys, secrets, or personal profile data. DID Core warns that public DID Documents can create privacy and correlation risk, so personal data belongs in credentials, peer-to-peer exchanges, or controlled service endpoints rather than the public document.

Identus developers use `did:prism` and `did:peer` for different jobs. `did:prism` fits public issuer and verifier identities that need ledger-backed resolution. `did:peer` fits relationship-specific DIDComm activity. A wallet, Cloud Agent, or mediator can use separate Peer DIDs to reduce correlation between relationships. The Identus DID management documentation describes Cloud Agent support for both managed PRISM DIDs and managed Peer DIDs.

DID Syntax and DID Methods

The generic syntax is:

```
did:<method>:<method-specific-id>
```

For `did:prism`, the PRISM DID method specification defines this shape:

```
did:prism:<initial-hash>[:<encoded-state>]
```

The short form contains the `did:prism:` prefix and a 64-character initial hash. The long form appends encoded initial state after another colon. PRISM uses the long form before the controller anchors the DID. After the controller publishes the DID operation and the PRISM VDR accepts it, software can use the short form as the published PRISM DID.

The encoded state in a long-form PRISM DID is not a JSON DID Document. It is method-specific state encoded by the PRISM method so a resolver can construct the initial DID Document before a public create operation appears in the VDR.

A **DID method** defines how software creates, resolves, updates, and deactivates DIDs and DID Documents for that method. DID Core defines the shared data model. The method specification defines the registry, protocol operations, encoding, validation rules, and lifecycle.

For `did:prism`, the PRISM method defines create, update, and deactivate operations. PRISM nodes read operations from the VDR and maintain the current state needed to construct DID Documents. The PRISM VDR specification describes SSI entries as event chains with create, update, and deactivate events.

DID Subjects and Controllers

The DID identifies the DID subject. The DID method authorizes the DID controller to change the DID Document. These roles can be the same entity, but they do not have to be.

For example, a university can be the subject of `did:prism:...`. The university can run an Identus Cloud Agent that holds the controller keys and publishes DID updates through PRISM Node. The university is still the issuer in the credential exchange; the Cloud Agent is infrastructure acting for the university.

The top-level `controller` property in a DID Document names one or more controller DIDs. DID Core treats this authorization separately from `authentication`. A verifier that checks an authentication proof checks the `authentication` relationship. Software that processes DID update authority follows the DID method's controller rules.

The `controller` property inside a `verificationMethod` has a different scope. It identifies the controller of that verification method. It can match the DID Document `id`, or it can point to another DID when a system delegates key control.

DID Document Structure

DID Documents share a common data model and can have different representations. DID Core and PRISM examples use JSON-LD with `@context`; JSON representations can omit JSON-LD context.

Common DID Document properties are:

Property	What software uses it for
`id`	The DID of the subject described by the DID Document.
`controller`	One or more DIDs authorized to control the DID Document under the DID method's rules.
`verificationMethod`	Public verification material, such as JWKs, plus an `id`, `type`, and method controller.
`authentication`	Verification methods approved for challenge-response authentication as the DID subject.
`assertionMethod`	Verification methods approved for making claims, including issuing Verifiable Credentials.
`keyAgreement`	Verification methods approved for key agreement, including DIDComm encryption setup.
`capabilityInvocation`	Verification methods approved for invoking object capabilities.
`capabilityDelegation`	Verification methods approved for delegating object capabilities.
`service`	Endpoints or service descriptors for interaction, such as DIDComm messaging.
`@context`	JSON-LD context used by JSON-LD representations.

Verification methods can be embedded directly in a verification relationship or referenced by DID URL. PRISM examples define methods once under `verificationMethod` and reference them from `authentication`, `assertionMethod`, or `keyAgreement`.

Example PRISM DID Document

This example uses fake, shortened values so the structure stays readable. Do not treat these identifiers or keys as resolvable PRISM data.

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/jws-2020/v1",
    "https://didcomm.org/messaging/contexts/v2"
  ],
  "id": "did:prism:example-university",
  "verificationMethod": [
    {
      "id": "did:prism:example-university#authentication-key-1",
      "type": "JsonWebKey2020",
      "controller": "did:prism:example-university",
      "publicKeyJwk": {
        "kty": "OKP",
        "crv": "Ed25519",
        "x": "fake-authentication-public-key"
      }
    },
    {
      "id": "did:prism:example-university#issuing-key-1",
      "type": "JsonWebKey2020",
      "controller": "did:prism:example-university",
      "publicKeyJwk": {
        "kty": "OKP",
        "crv": "Ed25519",
        "x": "fake-issuing-public-key"
      }
    },
```

```
    {
      "id": "did:prism:example-university#key-agreement-key-1",
      "type": "JsonWebKey2020",
      "controller": "did:prism:example-university",
      "publicKeyJwk": {
        "kty": "OKP",
        "crv": "X25519",
        "x": "fake-key-agreement-public-key"
      }
    }
  ],
  "authentication": [
    "did:prism:example-university#authentication-key-1"
  ],
  "assertionMethod": [
    "did:prism:example-university#issuing-key-1"
  ],
  "keyAgreement": [
    "did:prism:example-university#key-agreement-key-1"
  ],
  "service": [
    {
      "id": "did:prism:example-university#didcomm-1",
      "type": "DIDCommMessaging",
      "serviceEndpoint": [
        {
          "uri": "https://agent.example.edu/didcomm",
          "accept": ["didcomm/v2"],
          "routingKeys": ["did:peer:example-mediator#key-1"]
        }
      ]
    }
  ]
```

```
}
```

In the example, the issuer's software can use `#issuing-key-1` to sign a Verifiable Credential, and a verifier can check that the referenced key appears in `assertionMethod`. A DIDComm implementation can use `#key-agreement-key-1` to prepare encrypted messages and the `DIDCommMessaging` service to learn the accepted DIDComm profile, endpoint URI, and routing keys.

DID Lifecycle

Every DID method defines its own lifecycle rules. The common operations are create, resolve, update, and deactivate. Identus developers see those operations through Cloud Agent APIs, resolver APIs, SDK resolver configuration, and PRISM Node behavior.

For a managed PRISM DID in Cloud Agent, the controller creates a DID from a document template. Cloud Agent derives PRISM key material from a wallet seed and derivation path, stores the derivation path, and can reconstruct key material at runtime from the seed. The Identus Quick Start creates a long-form PRISM DID, publishes it through Cloud Agent, and then uses the short form as `publishedPrismDID` for schema and credential flows.

PRISM updates publish method operations that change the DID state, such as adding or removing verification methods or services. The VDR state machine treats deactivation as a final state. After deactivation, resolvers should no longer treat the DID as active.

For a managed Peer DID, Cloud Agent generates random key material and stores it in secret storage. Peer DIDs are not used as global public issuer identifiers. They fit DIDComm relationships and mediation, where each relationship can use separate identifiers and keys.

Resolvers

A resolver takes a DID and returns the DID Document plus metadata. The resolver must understand the DID method. A generic resolver can route different methods to method-specific drivers, but the method driver still performs the method-specific retrieval and validation.

For `did:prism`, a resolver needs the current PRISM state for the DID. Identus documents these resolution paths: Cloud Agent with PRISM Node, Universal Resolver support for PRISM DIDs, SDK resolver configuration, and community indexers. The Identus DID PRISM Resolver documentation describes Cloud Agent and PRISM Node as a solution for creating, updating, deactivating, and resolving PRISM DIDs.

Resolver choice is a trust decision. A verifier can use a hosted resolver, run its own resolver, or resolve against local indexed state. The verifier software should treat resolver output as input to verification, not as the whole trust decision. Credential verification still checks the proof, the verification relationship, credential status, schema, and verifier policy.

Controllers

Controllers are entities authorized to change the DID Document under the DID method's rules. A controller can be a person using a wallet, an organization running Cloud Agent, or a service acting under an organization's operational control. The DID Document records controllers as DIDs; the legal or governance authority behind those controllers lives outside the DID Document.

In `did:prism`, the controller proves authority through the method's signed operations. PRISM VDR validation rules require create operations to be signed with the DID's master key and update or deactivate events to be signed by a current master key. Cloud Agent-managed PRISM DID APIs perform this protocol work for the application, but the controller model still determines who can rotate keys, add services, or deactivate the DID.

Verification Flow

A verifier that checks a credential issued from a PRISM DID uses this flow.

1. The verifier receives a presentation from the holder's wallet or Edge Agent.
2. The verifier extracts the issuer DID and the verification method referenced by the credential proof.
3. The resolver resolves the issuer DID to a DID Document.
4. The verifier dereferences the verification method and checks that the method appears under `assertionMethod`.
5. The verifier checks the cryptographic proof with the public key in the verification method.
6. The verifier checks credential status, schema, and policy, including whether the issuer is trusted for the requested transaction.

DID verification relationships prevent key reuse across proof purposes. A key listed only under `authentication` cannot be used to issue a credential. A key listed only under `assertionMethod` cannot be used for DIDComm key agreement. The verifier software has to check the relationship that matches the operation it is validating.

Privacy Checks

Public DIDs are useful for issuers, verifiers, schemas, and trust registries, but they create correlation. A public issuer DID should remain stable so verifiers can resolve issuer keys and policy systems can refer to the issuer. A holder's wallet should use pairwise or relationship-specific DIDs for DIDComm connections unless the holder intends public correlation.

Before publishing a DID Document, the controller should check that it contains only the public material needed for the DID method and protocol flows:

1. No private keys, seeds, bearer tokens, credentials, or personal profile attributes.
2. No service endpoint URL that embeds a username, customer identifier, account number, or other correlating value.
3. No reused verification method across relationships intended to stay unlinkable.
4. No descriptive `type` or custom property that reveals the subject's category without a protocol need.

DIDs stay focused on public verification and routing. Credentials and presentations carry claims. Agents and wallets handle disclosure to specific verifiers.

Identus Concepts

[Identus Application Architecture Diagram]

Identus is made up of several open source components. Each could be used or forked separately but they are designed to work well together.

PRISM Node

PRISM Node implements the `did:prism` method and serves as a second-layer node for the Distributed Ledger, currently it only supports Cardano blockchain or local database but in the future it will be acting as a comprehensive interface to multiple VDR (Verifiable Data Registries). The node can resolve PRISM DIDs and write transactions to a blockchain or database, maintaining an indexed internal state that's synchronized with the underlying blockchain for efficient lookup operations.

As a critical component in the Identus ecosystem, PRISM Node provides a secure and trustworthy platform for storing and managing decentralized identifiers. It handles the creation, update, resolution, and deactivation of PRISM DIDs by generating transactions with the necessary operation information, verifying and validating these operations, and publishing them to the blockchain. Once transactions are confirmed, the node updates its internal state accordingly.

The PRISM Node's architecture enables users to: - Create unpublished DIDs via Apollo building block with the option to announce them publicly later. This means that not all DIDs are required to be published on a

VDR, for example, as a Holder you don't beed your DID to be public, but as an Issuer, the Cloud Agent will require the issuing DID to be public and anchored in a VDR. In this case, PRISM Node will handle that operation for you. - Update DID documents by publishing update operations on chain. - Deactivate DIDs by publishing deactivation operations on chain. - Resolving DIDs by querying historical changes on chain. - Track the status of operations submitted to the node.

This second-layer approach is essential for making DIDs scalable and efficient, as it provides the necessary off-chain processing and data storage capabilities while leveraging the security and immutability of the underlying blockchain. PRISM Node is expected to be online at all times to ensure reliable service.

Cloud Agent

Written in Scala, the Cloud Agent runs on a server and communicates with clients and peers via a REST API. It is a critical component of an Identus application, able to manage identity wallets and their associated operations, as well as issue Verifiable Credentials. The Cloud Agent is expected to be online at all times.

The Cloud Agent is designed to be scalable, robust, and standards-compliant, providing comprehensive self-sovereign identity services. It supports W3C standards, DIDCommV2, and Hyperledger Aries protocols, ensuring interoperability within the broader SSI ecosystem. Key capabilities include:

- Support for multiple agent roles including Issuer, Holder, Verifier
- Management of W3C Standard Verifiable Credentials (JSON and JSON-LD formats encoded as JWT), SD-JWT and AnonCreds
- Implementation of DIF Presentation Exchange for credential requests and submissions

- Support for `did:prism` and `did:peer` (version 2) DID methods
- Full implementation of DIDCommV2 messaging and protocols
- Compatibility with Hyperledger Aries RFCs including DID exchange, out-of-band protocol, issue credential, and present proof

The Cloud Agent's REST API enables developers to build controllers in any programming language without needing deep expertise in the underlying SSI standards. This architecture allows business logic to be separated from the identity infrastructure, making it easier to develop specialized applications while leveraging the full power of decentralized identity.

When deployed, the Cloud Agent interacts with the PRISM Node over gRPC protocol, using it as the Verifiable Data Registry to anchor DIDs on a distributed ledger for high security and availability.

Building Blocks

Identus separates the handling of important SSI operations into separate, focused libraries called "building blocks". These modular components can be combined and configured to meet various use cases and product requirements. This modular architecture provides excellent flexibility and customization options, allowing developers to implement decentralized identity solutions tailored to their specific needs.

Apollo - Cryptography

Apollo is a comprehensive cryptographic primitives toolbox that Identus uses to ensure data integrity, authenticity, and confidentiality. It provides the foundation for secure communication and data protection throughout the Identus ecosystem.

Apollo employs cryptographic hash functions to create digital fingerprints of data, allowing the detection of any unauthorized modifications. For example, when a verifier receives a credential, Apollo's cryptographic functions can verify that the credential hasn't been tampered with since it was issued.

Additionally, Apollo implements digital signatures to authenticate the identity of senders and recipients, and uses encryption algorithms to protect sensitive data from unauthorized access. In a healthcare scenario, Apollo's encryption would ensure that patient credentials remain confidential when shared between authorized parties.

Castor - DID

Castor enables the creation, management, and resolution of Decentralized Identifiers (DIDs). It currently supports the native `did:prism` method and the `did:peer` method but the are discussions aligning the team to build a more flexible architecture that would allow anyone to write plugins that could support other DID methods.

When a user creates a new digital identity in an Identus application, Castor generates the DID and associated cryptographic material. For instance, a university issuing student credentials would use Castor to create and manage DIDs for both the institution and potentially for each student, establishing the foundation for trusted digital relationships.

Castor's resolver component can look up a DID and retrieve its associated DID Document, which contains the public keys, authentication mechanisms, and service endpoints needed for secure interactions with that identity.

Pollux - Verifiable Credential

Pollux handles all Verifiable Credential operations, allowing users to issue, manage, and verify credentials in a privacy-preserving manner. This building block is essential for implementing the core functionality of credential exchange in self-sovereign identity systems.

In a real-world employment scenario, a company could use Pollux to issue employee credentials, which employees could then store in their digital wallets. When applying for a loan, an employee could selectively disclose relevant employment information from this credential without revealing unnecessary personal details. The bank could then use Pollux's verification capabilities to confirm the credential's authenticity and validity without contacting the employer directly.

Pollux also manages credential status, enabling issuers to revoke credentials when necessary and allowing verifiers to check if a credential is still valid before accepting it.

Mercury - DIDComm

Mercury provides an interface to the DIDCommV2 protocol, enabling secure, private communication between DIDs regardless of the underlying transport mechanisms. This building block establishes the foundation for all agent-to-agent communications in the Identus ecosystem.

For example, when a citizen wants to share a government-issued credential with a service provider, Mercury facilitates the encrypted, authenticated message exchange between the citizen's wallet and the service provider's verification system. This communication happens peer-to-peer, without requiring centralized intermediaries to facilitate the exchange.

Mercury's transport-agnostic design means that these secure communications can occur over various channels, including HTTP, WebSockets, or Bluetooth, making it versatile for different deployment scenarios from web applications to mobile devices.

More detailed information on each of the Building Blocks can be found in the Identus Documentation.

Edge Agent

Edge Agents bring agent capabilities to client applications such as websites, mobile apps, and other user-facing software. Unlike Cloud Agents, Edge Agents cannot be assumed to be online at all times, and therefore rely on sending and receiving all communications through an online proxy called a Mediator.

Edge Agents are implemented as SDKs that developers can integrate into their applications, providing a comprehensive suite of SSI functionality directly on the client side. These SDKs handle critical operations including:

- Creating and managing DIDs
- Storing and managing Verifiable Credentials
- Secure communication via DIDComm
- Cryptographic operations for signing and verification
- Local secure storage of identity data

Identus provides Edge Agent SDKs in multiple programming languages to support various platforms:

- **TypeScript SDK**: For web applications
- **Swift SDK**: For iOS applications
- **Kotlin Multiplatform SDK**: For Android applications and cross-platform development

Each SDK implements the same core building blocks (Apollo, Castor, Mercury, Pollux, and Pluto interfaces) as the Cloud Agent, ensuring consistent functionality and interoperability across the entire Identus ecosystem. This architectural consistency allows developers to create seamless experiences where identity operations can be performed either on the client or server side as appropriate for their use case.

Edge Agents typically function as digital wallets, enabling users to maintain control over their credentials and identity data on their personal devices.

This approach aligns with the core principles of Self-Sovereign Identity by keeping the user in control of their data and minimizing dependency on centralized services.

Mediator

Mediators act as middlemen between Peer DIDs. In order for any agent to send a message to any other agent, it must know the `to` and `from` DIDs of each message. The sender and recipient together make up a cryptographic connection called a `DIDPair`. Mediators maintain queues of messages for each `DIDPair`. If an Edge Agent is offline, the Mediator will hold incoming messages for them until the agent is back online and able to receive them. Mediators can deliver messages when polled, or push via web sockets. Mediators are expected to be online at all times and be highly available.

Identus officially supports and releases updates for their own Mediator implementation but there are a couple of alternative implementations close to the Identus community. In theory, any DIDCommV2 compatible mediator should work with Identus so this is not an exhaustive list.

- PRISM Mediator
- RootsID Mediator
- Blocktrust Mediator

Both RootsID and Blocktrust provide hosted instances of their mediators that are publicly accessible. While extremely helpful during development, these are not recommended for production Identus deployments as they have no uptime guarantee and will not scale past a small number of concurrent users. We will discuss how to run your own Mediator in Chapter .

Verifiable Data Registry (VDR)

The Verifiable Data Registry (VDR) addresses the critical challenge of ensuring authenticity and integrity for publicly available data in an environment where data is generated and consumed at unprecedented scales. Traditional systems often rely on centralized authorities, introducing single points of failure and undermining trust, especially when multiple independent parties need to rely on the same data.

Key challenges include enabling decentralized verification, guaranteeing data integrity and authenticity without central oversight, ensuring interoperability across diverse storage technologies (databases, blockchains, in-memory caches), and facilitating scalable, trustless collaboration.

The VDR system tackles these issues by providing a unified API and a modular, extensible framework for storing, mutating, retrieving, and removing data. It decouples the application layer from the underlying storage mechanisms through two main pluggable components:

- **Drivers**: These are plugins that implement the actual data storage and retrieval logic. Each driver is tailored to a specific storage backend (e.g., a database, a blockchain, or in-memory storage). Drivers are responsible for executing operations like creating, reading, updating, and deleting data. They also handle the generation and verification of cryptographic proofs (hashes for immutable data, digital signatures for mutable data) to ensure data integrity.
- **URL Managers**: These components construct and resolve URLs that reference the stored data. They embed necessary metadata (like driver identifiers, driver families, and cryptographic proofs) into these URLs using standard query parameters. This allows the VDR system to select the correct driver and verify data integrity when a URL is presented.

By abstracting these operations, the VDR layer ensures consistent and verifiable data operations regardless of where or how the data is stored. It employs cryptographic hashes for immutable data and digital signatures for mutable data, embedded in URLs or retrievable via the driver—to guarantee integrity. This architecture fosters a trustless environment where multiple parties can independently verify data authenticity and integrity, enhancing security and collaboration. The VDR system's design allows for adaptability to various storage backends while maintaining consistent methods for data verification and access.

Section II - Getting Started

Installation - Local Environment

Overview

Hyperledger Identus, previously known as Atala PRISM, is distributed across various repositories. These repositories group together different building blocks to provide the necessary functionality for fulfilling each of the essential roles in Self-Sovereign Identity (SSI), as introduced in Identus Concepts and SSI Basics. Throughout this book, we will detail the setup of each component.

The initial component to set up is our Cloud Agent. This agent is responsible for creating and publishing DID Documents into a Verifiable Data Registry (VDR), issuing Verifiable Credentials, and, depending on the configuration, even providing Identity Wallets to multiple users through a multi-tenancy setup. For now, our focus will be on setting up the Cloud Agent to run locally in development mode, supporting only a single tenant. This step is crucial for learning the basics and getting started. As we progress and build our example application, we will deploy the Cloud Agent in development mode first, then set it up to connect to Cardano `testnet` as our VDR on pre-production mode and, finally, in production mode with multi-tenancy support connected to Cardano `mainnet`. This will be a gradual process as we need to familiarize on each stage of the Cloud Agent.

Identus Releases Overview

Identus is built upon multiple interdependent building blocks, including the Cloud Agent, Wallet SDK, Mediator, and Apollo Crypto Library. To ensure compatibility among these components, it is crucial to identify the correct building block versions that are compatible between them. Identus releases are listed here. The release notes for each vesion provides a compatibility table for each Identus release. This guide will focus on the latest stable Identus release at the time of writing (December 2024).

Pre-requisites

Git

If you're using a UNIX-based system (such as OS X or Linux), you likely already have `git` installed. If not, you can download the installer from Git downloads. Additionally, various GUI clients are available for those who prefer a graphical interface.

To clone the Cloud Agent repository, first go to the Releases page and identify the tagged release corresponding to the Identus release you are targeting (e.g. `cloud-agent-v1.40.0` is part of Identus v2.14 release), then clone the repository with this command:

```
git clone --depth 1 --branch cloud-agent-v1.40.0 https://github.com/hyperledg
```

> ℹ Note
>
> Using `--depth 1` will skip the history of changes in the repository up until the point of the tag, this is optional.

Docker

The Cloud Agent and Mediator are distributed as Docker containers, which is the recommended method for starting and stopping the various components required to run the cloud infrastructure.

To begin, install Docker Desktop, which provides everything you need to get started.

WSL

For Windows users, please refer to How to install Linux on Windows with WSL.

> **ℹ Note**
>
> Windows is the least tested environment, the community has already found some issues and workarounds on how to get the Cloud Agent working. We will try to always include instructions regarding this use case.

Before We Run The Agent

Once you have cloned the `identus-cloud-agent` repository and Docker is up and running you can jump right ahead and run the agent, but before we do that if you are not yet familiar with the community projects or the structure of the agent itself, we recommend you to spend a little time exploring the following information, this is optional and you can skip it.

Atala Community Projects

There is a growing list of community repositories that aim to provide some extra functionality, mostly maintained by official developers and community members on their spare time.

Some notable projects are:

- **Pluto Encrypted:** Implementation of Pluto storage engine with encryption support.
- **Identus Store** A secure light-weight and dependency free database wrapper.
- **NeoPrism Resolver:** A did:prism resover and explorer.
- **Blocktrust Mediator** A DIDCommv2 compliant mediator, written in C#.
- **Edge Agent SDK Demos:** Browser and Node versions of Edge Agent SDK integrated with Pluto Encrypted.
- **Identus Test:** Shell script helper that will checkout a particular Identus release and compatible components.

Exploring The Repository

There are two fundamental directories inside the repository if you are an end user.

1. `docs` Where all the latest *technical documentation* will be available, this includes the Architecture Decision Records (ADR), general insights, guides to deploy, examples and tutorials about how to handle VC Schemas, Connections, secrets, etc. We will do our best to explain in detail all this procedures as we build our example app.
2. `infrastructure` This directory holds the agent's Docker file and related scripts to run the agent in different modes such as `dev`, `local` or `multi`. **The way to change the agent setup is by customizing environmental variables trough the Docker file**,

> so we really advise you to get familiar with the `shared` directory content, because that's the base for every other mode, in essence, every mode is a customization of the shared Docker file.

Our first mode to explore and the simplest one should be `local` mode, which by default will run a single agent as a single-tenant, meaning that this instance will control only a single Identity Wallet that will be automatically created and seeded upon the first start of the agent.

The `multi` mode essentially runs 3 different `local` agents but each is assigned a particular role such as `issuer`, `holder` and `verifier`. This is useful in order try test more complex interactions between independent actors.

Finally the `dev` mode is meant to be used for development and provides an easy way to modify the Cloud Agent source code, it does not rely on the pre-built Docker images that the `local` mode fetches and run. We will not use this mode at all trough the book but feel free to explore this option if you would like to make contributions to the Cloud Agent in the future.

Running The Cloud Agent

Environment Variables

Inside `infrastructure/local` directory, you will find three important files, `run.sh`and `stop.sh` scripts and the `.env` file.

Our `local` environment file should look like this

```
AGENT_VERSION=1.40.0
PRISM_NODE_VERSION=2.5.0
VAULT_DEV_ROOT_TOKEN_ID=root
```

This will tell Docker which versions of the Cloud Agent and PRISM Node to run, plus a default value for the `VAULT_DEV_ROOT_TOKEN_ID`, this value corresponds to the HashiCorp Token ID. HashiCorp is a secrets storage engine and it will become relevant later on when we need to prepare the agent to run in `prepod` and `production` modes, for now the `local` mode will ignore this value because by default it will use a local `postgres` database for it's secret storage engine.

The `run.sh` script options:

```
./run.sh --help
Run an instance of the ATALA bulding-block stack locally

Syntax: run.sh [-n/--name NAME|-p/--port PORT|-b/--background|-e/--env|-w/--w
options:
-n/--name              Name of this instance - defaults to dev.
-p/--port              Port to run this instance on - defaults to 80.
-b/--background        Run in docker-compose daemon mode in the background.
-e/--env               Provide your own .env file with versions.
-w/--wait              Wait until all containers are healthy (only in the bac
--network              Specify a docker network to run containers on.
--webhook              Specify webhook URL for agent events
--webhook-api-key      Specify api key to secure webhook if required
--debug                Run additional services for debug using docker-compose
-h/--help              Print this help text.
```

For our first interaction with the agent all we have to do is to call the run script. If you have any conflicts with the port 80 already in use or you don't want that as the default, you can pass `--port 8080` or any other available port that you would like to use.

So, from the root of the repository you can run:

```
./infrastructure/local/run.sh
```

This will take a while the first time as Docker will fetch the required container images and get them running. To check the status of the Cloud Agent you can use `curl` or open a browser window at same endpoint URL (make sure to specify a custom port if you changed it in the previous step, e.g. use `http://localhost:8080`):

```
curl http://localhost/cloud-agent/_system/health
{"version":"1.40.0"}
```

The `version` should match the version of the Cloud Agent defined in the `.env`file.

To stop the agent, you can press `Control + C` or run:

```
./infrastructure/local/stop.sh
```

Congratulations! you have successfully setup the agent in `local` mode. Next we will explore our Docker file in detail and interact with our agent using the REST API.

Agent REST API

The Cloud Agent API

The local Cloud Agent exposes two public protocol surfaces through APISIX. Your controller application uses the REST API to create DIDs, register credential schemas, issue Verifiable Credentials, manage connection records, request presentations, and inspect protocol state. Other agents and wallets use the DIDComm endpoint to deliver encrypted agent messages.

Those two surfaces share the same wallet state. A REST request changes or reads records in the Cloud Agent wallet. A DIDComm message may change the same records after the Cloud Agent processes an inbound connection, issuance, or presentation message. The Cloud Agent README describes this pattern as a controller that sends HTTP requests to the agent and processes webhook notifications from it.

The local stack from the previous chapter runs in single-tenant mode. In that mode, the Cloud Agent uses the default entity and the default wallet for REST API and DIDComm operations. The Cloud Agent authentication documentation defines both IDs as `00000000-0000-0000-0000-000000000000`.

You can confirm that default wallet initialization in the container logs:

```
docker logs local-cloud-agent-1 | grep "default"
```

```
... Initializing default wallet.
... Default wallet seed is not provided. New seed will be generated.
... Entity created: Entity(00000000-0000-0000-0000-000000000000,default,00000
```

Later, the example application will use multi-tenant mode. In multi-tenant mode, the Cloud Agent serves multiple tenants from one shared Cloud Agent instance. The multi-tenancy documentation defines an entity as the tenant representation and a wallet as the isolated container for that tenant's DIDs, connections, credentials, keys, credential schemas, and related assets. A service-managed wallet fits a server-side issuer, verifier, or custodial wallet service where the operator manages the runtime, database, secret storage, API access, and backups.

The `_system/health` endpoint from the previous chapter reports the running service version. The system API exposes one more endpoint for runtime metrics:

```
curl http://localhost/cloud-agent/_system/metrics
# HELP jvm_memory_bytes_used
# TYPE jvm_memory_bytes_used gauge
jvm_memory_bytes_used{area="heap",} 1.07522616E8
jvm_memory_bytes_used{area="nonheap",} 1.74044936E8
...
```

The Cloud Agent OpenAPI specification describes the metrics endpoint as Prometheus text output from the internal metrics registry. Use it when you need local runtime data for memory, latency, or health investigations.

OpenAPI Specification

The OpenAPI Specification (`OAS`) is a standard description format for HTTP APIs. It gives humans and tools a shared contract for paths, request bodies, response bodies, authentication schemes, and schemas. The Cloud Agent source repository includes an OpenAPI 3.1 document titled `Identus Cloud Agent API Reference`, and the local Docker stack exposes that document through APISIX.

For the local agent started in the previous chapter, use this URL as the version-matched API contract:

```
http://localhost/docs/cloud-agent/api/docs.yaml
```

If you started the agent with a different `run.sh --port` value, replace `localhost` with `localhost:<port>`. Swagger UI reads this same document. Postman and client generators can import it to create a collection or client stub for the exact Cloud Agent version you are running.

The hosted Identus docs provide current tutorials and conceptual material at https://hyperledger-identus.github.io/docs/. For endpoint-level work in this local stack, prefer the OpenAPI document served by your running agent. It avoids mismatches between the book, a hosted page, and the Docker image version.

APISIX Gateway

APISIX proxies the Cloud Agent services inside the local Docker stack. The `run.sh` script binds APISIX to the port you selected, with `80` as the default. The Cloud Agent's shared APISIX configuration defines these routes:

Local route	Upstream service	Purpose
http://localhost/cloud-agent/	`cloud-agent:8085`	REST API for controller applications.

Local route	Upstream service	Purpose
http://localhost/docs/cloud-agent/api/docs.yaml	`cloud-agent:8085`	OpenAPI document for the running Cloud Agent.
http://localhost/apidocs/	`swagger-ui:8080`	Swagger UI for interactive API calls.
http://localhost/didcomm/	`cloud-agent:8090`	Public DIDComm endpoint for encrypted agent messages.

The DIDComm endpoint is the transport endpoint advertised in invitations and DID documents. A peer uses it to send encrypted DIDComm messages to this Cloud Agent. The DIDComm chapter covers the message model and protocol flow later in the book.

APISIX routes match incoming requests, apply route plugins, and forward requests to upstream services. The Cloud Agent local configuration uses `proxy-rewrite` to strip the public route prefix before forwarding the request, and it enables the APISIX `cors` plugin on the REST and DIDComm routes.

> ⚠ Warning
>
> The local APISIX CORS configuration allows all origins on the `/cloud-agent/*` and `/didcomm*` routes. That setting is convenient for local browser testing. A deployed service should restrict allowed origins to the domains that host the controller application, admin tools, or wallet front ends that need browser access.

Swagger UI

Swagger UI renders API documentation and an interactive request form from an OpenAPI document. The local Docker stack includes a Swagger UI container, and APISIX exposes it at http://localhost/apidocs/.

Open http://localhost/apidocs/ in your browser. In the server list, select:

- `http://localhost/cloud-agent - The local instance of the Cloud Agent behind the APISIX proxy`.

Click `Authorize`. Swagger UI opens a modal for the `apikey` header. The local stack has API key authentication disabled by default, so the Cloud Agent will accept the request even if Swagger UI sends any value. Use `test` for now. In later chapters, after the book enables API key authentication, this field must contain the API key assigned to the default entity or tenant entity.

After authorizing, the modal should look like this:

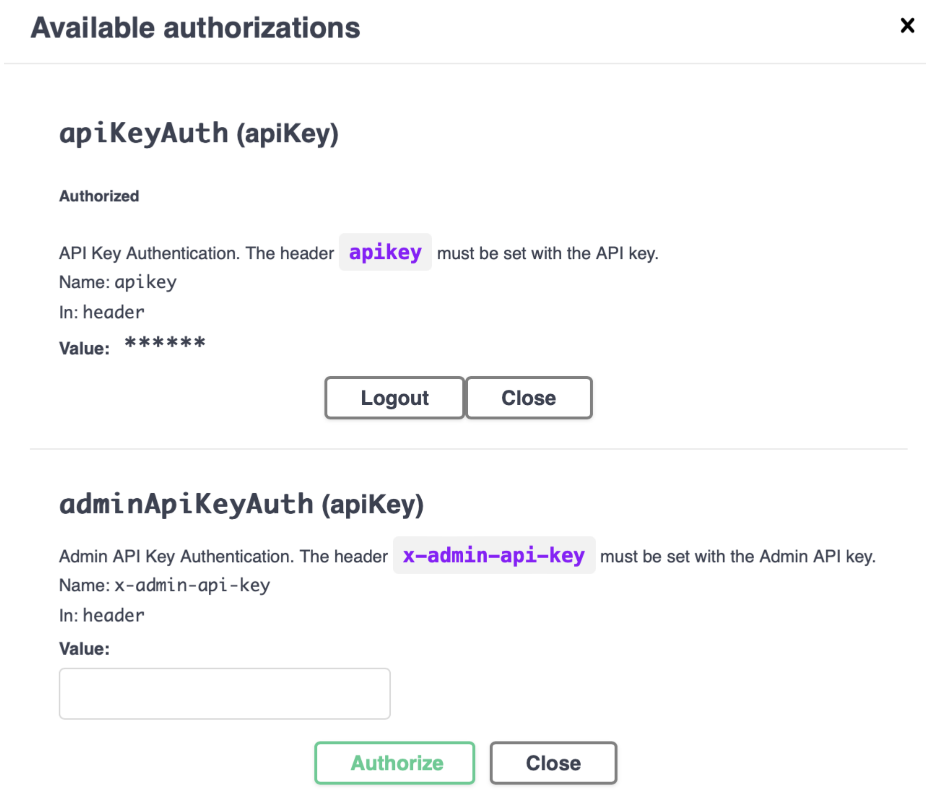

Figure 1: Swagger UI Apikey Modal

Close the modal and try the first request. Expand `GET /connections` and click `Try it out`. Leave `offset`, `limit`, and `thid` blank. Click `Execute` to send the request.

The response should look similar to this:

```
{ "contents": [], "kind": "ConnectionsPage", "self": "", "pageOf": "" }
```


An empty `contents` array means the default wallet has no connection records yet. That is the expected result for a new local agent.

> **Note**
>
> Swagger UI can generate a `curl` command for the request it sends. You can paste that command in your terminal to run the same API call outside the browser. For example:
>
> ```
> curl -X 'GET' \
>   'http://localhost/cloud-agent/connections' \
>   -H 'accept: application/json' \
>   -H 'apikey: test'
> {"contents":[],"kind":"ConnectionsPage","self":"","pageOf":""}
> ```

Postman

Swagger UI is useful for quick checks against the running agent. Postman is better when you need saved environments, request variables, scripted assertions, shared collections, or repeated manual testing across issuer, holder, and verifier agents.

Postman can import OpenAPI definitions from a file, a URL, pasted YAML or JSON, or a repository. Use the local Cloud Agent OpenAPI URL so the collection matches your running Docker image.

1. Install Postman if you do not already have it.
2. Copy the local OpenAPI URL: `http://localhost/docs/cloud-agent/api/docs.yaml`. If you changed the `run.sh --port` value, include that port in the URL.

3. In Postman, go to `File -> Import`.
4. Paste the OpenAPI URL in the import box, or download the YAML from that URL and import the file.
5. Import the definition as a collection.

After import, Postman should show a collection named `Identus Cloud Agent API Reference`. Set any collection base URL or server variable to `http://localhost/cloud-agent` if Postman does not select that server for you.

Tutorials

The official Identus tutorials cover the main API flows: connections, DID management, credential schemas, credential issuance, presentation requests, webhooks, multi-tenancy, and VDR interaction. Use those tutorials as the endpoint reference for the next chapters. This book will connect those API operations to the example application, the wallet model, DIDComm flows, and verifier policy checks.

Mediator

Overview

The `mediator` is a service that enables private and secure message routing between peers. Its primary function is to relay encrypted messages between agents using DIDComm V2 set of communication protocols. Since each participant wallet is Self-Sovereign, mediators help route messages to their intended recipient without any details of each peer besides the agreed public keys over the initial connection protocol, upon which they both create a `did:peer`. Encrypted messages can be sent to peers even if they are not online by relaying on the mediator to manage a queue for the offline party until they are able to connect and retrieve them.

Think of a mediator like a set of Post Office mailboxes. Each mailbox has a mailing address (*a `DID`*). Each mailbox can hold an stack of incoming mail Encrypted DIDComm V2 messages for its owner. Each mailbox also keeps a list of cryptographic Connections with whom its owner can send or receive mail with. In this way, Identus mediators facilitate all types of important transactions, including routing both messages, Verifiable Credential issuance, Credential Exchange or any custom message for your own use.

The key insight for this piece of infrastructure is that each agent can define their own `mediator` to be reachable at, so whenever they establish a `did:peer` with another agent, each peer defines their own reachable `DIDCOMM` endpoint, each one will tell the other their `mediator` as the way to send messages addressed to them.

Why use a Mediator?

Why do we need a `mediator` in the first place? If participants are Self-Sovereign, then why not connect an edge wallet directly to another edge wallet?

SSI participants are connected through various types of devices and network conditions. The Cloud Agent, is expected to be online and available all the time, as it's usually hosted in a cloud infrastructure somewhere. However users who keep their wallets on mobile devices, or laptop computers are only online sporadically and can't be expected to have a reliable connection to the Internet at all times. This is where mediators earn their keep. If `Alice` were to send a message to `Bob`, who happens to be offline at the moment, the message could not be delivered, plus `Alice` would also have to know sensitive data about how to contact `Bob`. If `Bob` changes devices, they would have to tell `Alice`, and all other peer connections. If `Alice` instead, sends the message through the `mediator`, it can be held securely and retrieved the next time `Bob` connects to the Internet. Mediators provide reliability between Connections, without compromising privacy or security.

Set up a Mediator

As usual, we use Docker to setup our infrastructure, `mediator` is no different and here is an example `docker-compose.yml`, `mongo-initdb.js` and `.env` files, you should put all 3 files in a folder.

`docker-compose.yml`

```
services:
  ################
  ### MEDIATOR ###
  ################
```

```yaml
  mediator-mongo:
    image: mongo:6.0
    command: [ "--auth" ]
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin
      - MONGO_INITDB_ROOT_PASSWORD=admin
      - MONGO_INITDB_DATABASE=mediator
    volumes:
      - ./mongo-initdb.js:/docker-entrypoint-initdb.d/initdb.js

  mediator:
    image: docker.io/identus/identus-mediator:${MEDIATOR_VERSION:latest}
    ports:
      - "${MEDIATOR_PORT:-8080}:8080"
    environment:
      # Creates the identity:
      # These keys are for demo purpose only for production deployments generate keys
      # Please follow the README file in the Mediator repository for guidelines on How to ge
      # KEY_AGREEMENT KEY_AUTHENTICATION are using format JOSE(JWK) OKP type base64urlsafe e
      - KEY_AGREEMENT_D=${KEY_AGREEMENT_D:-Z6D8LduZgZ6LnrOHPrMTS6uU2u5Btsrk1SGs4fn8M7c}
      - KEY_AGREEMENT_X=${KEY_AGREEMENT_X:-Sr4SkIskjN_VdKTn0zkjYbhGTWArdUNE4j_DmUpnQGw}
      - KEY_AUTHENTICATION_D=${KEY_AUTHENTICATION_D:-INXCnxFEl0atLIIQYruHzGd5sUivMRyQOzu87qV
      - KEY_AUTHENTICATION_X=${KEY_AUTHENTICATION_X:-MBjnXZxkMcoQVVL21hahWAw43RuAG-i64ipbeKK
      - SERVICE_ENDPOINTS=${SERVICE_ENDPOINTS:-http://identus-mediator:8080;ws://identus-med
      - MONGODB_USER=admin
      - MONGODB_PASSWORD=admin
      - MONGODB_PROTOCOL=mongodb
      - MONGODB_HOST=mediator-mongo
      - MONGODB_PORT=27017
      - MONGODB_DB_NAME=mediator
    depends_on:
      - "mediator-mongo"
```

mongo-initdb.js

```
db.createUser({
    user: "admin",
    pwd: "admin",
    roles: [
        { role: "readWrite", db: "mediator" }
    ]
});

const database = 'mediator';
const collectionDidAccount = 'user.account';
const collectionMessages = 'messages';
const collectionMessagesSend = 'messages.outbound';

// The current database to use.
use(database);

// Create  collections.
db.createCollection(collectionDidAccount);
db.createCollection(collectionMessages);
db.createCollection(collectionMessagesSend);

//create index
db.getCollection(collectionDidAccount).createIndex({ 'did': 1 }, { unique: tr
// Only enforce uniqueness on non-empty arrays
db.getCollection(collectionDidAccount).createIndex({ 'alias': 1 }, { unique:
db.getCollection(collectionDidAccount).createIndex({ "messagesRef.hash": 1, '

// There are 2 message types  `Mediator`  and `User` Please follow the Readme
const expireAfterSeconds = 7 * 24 * 60 * 60; // 7 day * 24 hours * 60 minutes
db.getCollection(collectionMessages).createIndex(
    { ts: 1 },
    {
```

```
        name: "message-ttl-index",
        partialFilterExpression: { "message_type": "Mediator" },
        expireAfterSeconds: expireAfterSeconds
    }
)
```

`.env`

```
### MEDIATOR
MEDIATOR_PORT=127.0.0.1:8080
MEDIATOR_VERSION=1.0.0
KEY_AGREEMENT_D=xxx
KEY_AGREEMENT_X=xxx
KEY_AUTHENTICATION_D=xxx
KEY_AUTHENTICATION_X=xxx
SERVICE_ENDPOINTS="https://example.com"
```

Your `.env` file will hold all the important bits, lets trough them.

`MEDIATOR_PORT` will define which port the container will be reachable at, I have also bound it to the local interface because I want to proxy this service over SSL with `apache` or `nginx`.

`MEDIATOR_VERSION` will default to `latest` but you can also define a specific version, this is important because sometimes there are breaking changes and your `cloud-agent` or SDK will be compatible with a specific version of the `mediator` that may not be the latest.

`KEY_AGREEMENT_D`, `KEY_AGREEMENT_X`, `KEY_AUTHENTICATION_D`, `KEY_AUTHENTICATION_X` are all the secret keys that are needed for the mediator to generate `DIDs`, in this case the `mediator` generates a `did:peer` so others can route messages trough it. Follow the latest updated guide to generate keys.

`SERVICE_ENDPOINTS` is how you define the *public url* of your `mediator`, this is very important, this is how other agents will reach this service, it can be the public IP or domain, you can include specific ports as well. Make sure `MEDIATOR_PORT` and `SERVICE_ENDPOINTS` are in agreement, for example, if you bound `MEDIATOR_PORT` to the local interface and your `SERVICE_ENDPOINTS` defines a public URL, you need to proxy your local port trough `apache`, `nginx` or similar, if you decide to use expose a public IP, all you need to do is to define both to the same IP and port. This is all contextual of how you run your infrastructure.

Once your done setting the environment you can run it with:

```
docker compose up -d
```

Your mediator should be reachable trough the same endpoint defined in `SERVICE_ENDPOINTS`.

If successful, you should see a simple web page with an invite and a QR code for your `mediator`.

Section III - Building

Example Project

Overview - Airline Ticket Wallet

One of the best ways to learn and understand a software platform is to build something with it. In the following chapters, we will build an example application showing how to use Identus to issue and verify airline tickets.

Users will be able to purchase a flight, and receive a Verifiable Credential representing a ticket and seat assignment. Travelers will be able to present their ticket to Airport Security when requested, and in turn, the security officer will be able to verify the ticket's authenticity.

We will use the Identus Cloud Agent, plus the Typescript SDK in our examples, however the same functionality and source is available in the native language of each Edge Agent SDK; Swift for iOS/Mac and Kotlin for Android. Please see the book's Github page for the complete source code and follow along in your language of choice.

Prerequisites

To follow along with the book, please make sure you have a working Cloud Agent development environment, as described in the Section 2.

Roles

Issuer: Airline

Holder: Traveler

Verifier: Airport Security Officer

Wallets

Overview

An SSI wallet is software that stores and protects the material a holder, issuer, or verifier needs to act in an identity protocol. In the W3C Verifiable Credentials Data Model 2.0, a credential repository stores and protects access to a holder's credentials. In W3C DID Core, a DID resolves to a DID document that can expose public verification methods and service endpoints. The private keys, credentials, DIDComm records, and local wallet state live in wallet storage, not in the DID document.

Identus uses the word wallet in two common places:

- An Edge Agent wallet runs inside a holder-facing application, such as a browser app, mobile app, or desktop app. The application embeds an Identus Edge SDK and provides storage through Pluto.
- A Cloud Agent wallet is a server-side wallet container accessed through the Cloud Agent REST API. A controller or tenant uses API calls and webhooks rather than direct in-process SDK calls.

Do not treat a ledger, PRISM Node, or DID resolver as a wallet backup. Public DID state can help other parties resolve issuer keys and service endpoints. It cannot reconstruct holder-held credentials, Peer DID private keys, DIDComm message history, connection records, or presentation state. A production wallet needs an explicit storage, encryption, backup, and recovery design.

For the airline ticket example in this section, the airline runs Cloud Agent as an issuer. The traveler uses an Edge Agent wallet as the holder. Airport security can use Cloud Agent, an Edge SDK, or another verifier implementation, depending on the deployment.

Wallet Responsibilities

A wallet has more work to do than storing credentials. It owns the local state needed to continue protocol flows across app restarts, device changes, and network interruptions.

The wallet stores or references:

- DID records and DID metadata.
- Private keys, wallet seeds, or secret references used by DID operations.
- Verifiable Credentials received from issuers.
- Presentation records created for verifiers.
- DIDComm messages, connection records, and protocol state.
- Mediator configuration for offline delivery.
- Credential status and schema references needed by later verification flows.

The wallet uses that state to perform protocol work. The holder wallet receives credential offers, asks the holder for consent, stores issued credentials, creates presentations, and sends DIDComm responses. The issuer wallet signs credentials and tracks issuance records. The verifier wallet or verifier agent receives presentations, checks cryptographic proofs, resolves issuer material, checks credential status, and then applies verifier policy.

Keep the verification layers separate. A cryptographic check answers whether the presentation, credential proof, issuer key, holder binding, and status data verify. A policy check answers whether the verifier accepts that issuer, credential type, schema, subject, freshness, or assurance level for the business action.

Edge Agent Wallets

Identus Edge Agent SDKs let an application run wallet and agent capabilities close to the holder. Current Identus repositories provide:

- TypeScript SDK for browser and Node.js applications. The current TypeScript SDK docs name the package `@hyperledger/identus-sdk` and note the rename from `@hyperledger/identus-edge-agent-sdk`.
- Swift SDK for Apple platforms.
- Kotlin Multiplatform SDK for Android and JVM applications.

The SDKs use the same architectural vocabulary:

- Apollo provides cryptographic operations.
- Castor creates, manages, and resolves DIDs.
- Pollux handles credential operations in the Swift and Kotlin SDKs. The current TypeScript SDK exposes credential capabilities through its modules and plugins.
- Mercury handles DIDComm v2 messages and related secure messaging work.
- Pluto provides the storage interface.
- Agent or EdgeAgent composes the building blocks into a higher-level agent API.

Start with Agent or EdgeAgent for application code. Direct calls into Apollo, Castor, Mercury, Pollux, or Pluto fit cases where the application needs a lower-level operation that the agent API does not expose. For example, a wallet that implements a custom DIDComm protocol can build and pack messages through Mercury, then store the resulting protocol records through the same wallet storage layer.

Pluto Storage

Pluto is the key implementation boundary in an Edge Agent wallet. Identus defines the storage interface, and the application provides the storage implementation that matches its platform and threat model.

For a browser wallet, the implementation can use IndexedDB or another browser database. For a mobile wallet, it can use the platform database plus Keychain or Keystore-backed key protection. For a server-side test wallet, an in-memory store works for a disposable run. Production storage should encrypt sensitive data at rest, bind encryption keys to the platform security model where possible, and support tested backup and restore.

Storage design changes wallet behavior. If the application stores credentials but loses private keys, the holder can lose the ability to prove control of the DID used during issuance or presentation. If the application stores keys but loses credentials, the holder can still control the DID but no longer has the credential payload. If the application loses DIDComm protocol records, it can fail to resume issuance, connection, or proof flows after restart.

Treat community Pluto stores as dependencies to evaluate, not as Identus defaults. Check maintenance status, encryption behavior, migration support, and compatibility with the SDK version used by the application before selecting one.

Cloud Agent Wallets

Cloud Agent runs on a server and exposes wallet operations through REST APIs and webhook-driven workflows. The Cloud Agent README describes it as a W3C and Aries standards-based cloud agent that supports DIDComm v2, credential issuance, credential verification, credential holding, secret management, and multi-tenancy.

A Cloud Agent wallet fits services that need server-side identity operations. An airline can use Cloud Agent to manage issuer DIDs, sign ticket credentials, send offers, and track issuance state. An airport verifier can use Cloud Agent to request presentations and receive verification results. A company can run Cloud Agent in multi-tenant mode when each customer or department needs a separate wallet.

Cloud Agent custody has a different trust boundary from an Edge Agent wallet. The tenant or business controller interacts with the wallet through an API. The service operator controls the runtime, database, secret storage, networking, and operational access. A service-managed wallet can be a good fit for organizations and delegated user wallets, but the deployment must define who can administer wallets, export data, rotate secrets, recover seeds, and inspect logs.

Multi-Tenant Wallets

In Cloud Agent multi-tenancy, an administrator creates a wallet, creates an entity for the tenant, and provisions an authentication method for that entity. The Identus tenant onboarding documentation describes the wallet as the container for tenant assets, including DIDs, credentials, and connections. After provisioning, Cloud Agent scopes tenant API calls to that wallet.

This model fits service-managed wallets. It does not remove custody risk. The administrator API, tenant authentication, wallet identifiers, database rows, Vault paths, and webhook routing must all preserve tenant separation.

Cloud Agent Key Material

Identus Cloud Agent uses different key-management paths for PRISM DIDs and Peer DIDs.

For managed PRISM DIDs, Cloud Agent derives key material from a wallet seed and derivation path. The DID management documentation states that Cloud Agent stores the derivation path rather than the PRISM key material itself, then reconstructs key material at runtime from the seed and path.

For managed Peer DIDs, Cloud Agent generates key material and stores it in secret storage. Peer DIDs support DIDComm activity, so these keys protect relationship-specific communications.

The Cloud Agent secrets storage documentation names HashiCorp Vault as the default secret storage service. It describes secret types such as seeds, private keys, credential-definition data, and AnonCreds link secrets. In multi-tenant configuration, Vault asset paths include the wallet ID so each wallet's secrets live under its own path.

Choosing Edge or Cloud Custody

Use an Edge Agent wallet when the holder should keep credentials and keys in a user-controlled application. This model fits traveler wallets, employee wallets, citizen wallets, and mobile proof presentation. The application team must own storage, encryption, backup, recovery, mediator use, and local user consent.

Use a Cloud Agent wallet when a server must act for an organization or delegated tenant. This model fits issuers, verifiers, service-managed organizational wallets, and backend workflows that need REST APIs and webhooks. The deployment team must own secret storage, authentication, tenant separation, monitoring, and operational controls.

Many Identus systems use both. The issuer and verifier run Cloud Agent. The holder uses an Edge Agent wallet. DIDComm, credentials, DIDs, and presentation protocols connect the two custody models without requiring the same wallet implementation on every side.

Airline Ticket Flow

The airline ticket example uses the wallet boundary in a concrete way:

1. The airline creates or selects a Cloud Agent wallet for issuer operations.
2. The airline creates an issuer `did:prism` with an `assertionMethod` verification method and publishes the DID when verifiers need ledger-backed resolution.
3. The traveler opens an Edge Agent wallet. The wallet creates or selects holder DIDs and stores local secret material through Pluto.
4. The airline sends a ticket credential offer to the traveler wallet through DIDComm or another supported issuance path.
5. The traveler wallet shows the offer, records holder consent, requests the credential, receives it, and stores it.
6. Airport security requests proof of ticket status, flight, passenger binding, or another accepted claim set.
7. The traveler wallet creates a presentation from the stored credential and sends it to the verifier.
8. The verifier software checks the presentation proof, issuer DID resolution, credential status, holder binding, schema, and format-specific rules.
9. Airport security policy decides whether the verified result satisfies the checkpoint rule.

Steps 8 and 9 are separate decisions. The verifier software can report that the credential proof is valid, the issuer DID resolved, and the credential is active. Airport policy can still reject the presentation if the flight date, airport, issuer, credential type, or passenger binding does not match the checkpoint rule.

Implementation Notes

For Edge Agent wallets:

- Pick the SDK for the host platform before designing storage.
- Treat Pluto as a required wallet component, not an optional cache.
- Encrypt wallet contents that include credentials, DIDComm records, seeds, private keys, or secret references.
- Test restore on a clean device or browser profile.
- Use `did:peer` for relationship-specific DIDComm activity and `did:prism` for public resolution.

For Cloud Agent wallets:

- Decide whether the deployment uses a default wallet or multi-tenancy before onboarding users.
- Keep administrator credentials separate from tenant credentials.
- Place wallet seeds and private key material in the configured secret storage.
- Back up the Cloud Agent database and secret storage as one recovery unit.
- Audit webhook delivery, tenant API keys, wallet IDs, and Vault paths during tenant onboarding.

A team completes wallet work only after it tests restore and protocol resumption. A wallet that can issue or receive credentials during a happy-path demo can still fail in production if the holder changes devices, the browser storage is cleared, the mediator queue grows, or a Cloud Agent tenant needs recovery after database restore.

DIDComm

Overview

DIDComm (*Decentralized Identifier Communication*) is a crucial component in the Self-Sovereign Identity (SSI) ecosystem, providing the secure messaging layer that enables Decentralized Identifiers (DIDs) to interact. The current specification, **DIDComm v2**, evolved from earlier iterations to address the growing needs for privacy, security, and interoperability in digital identity communications. Its fundamental purpose is to enable secure, private, and authenticated communication between entities identified by DIDs, regardless of their underlying infrastructure or technology stack, transforming DIDs from static identifiers into dynamic communication endpoints capable of engaging in rich interactions.

Within the SSI technology stack, DIDComm sits above the foundational DID layer (which provides identifiers and cryptographic material) and below the application-specific protocols that define particular interactions like credential exchange. This positioning makes DIDComm the connective tissue of the SSI interactions in Identus, enabling all higher-level identity interactions.

Key Characteristics

DIDComm's design incorporates several distinctive characteristics that make it particularly well-suited for decentralized identity communications:

Transport agnosticism allows DIDComm messages to travel over virtually any communication channel—HTTP(S), WebSockets, Bluetooth, NFC, mesh networks or even QR codes. This flexibility means that identity interactions aren't tied to specific network infrastructures, enabling truly peer-to-peer communications even in offline or intermittent connectivity scenarios.

End-to-end encryption ensures that message contents remain confidential between the sender and intended recipient(s), even when traversing untrusted networks or intermediaries. Using public key cryptography derived from DIDs, DIDComm creates secure communication channels that protect sensitive identity information.

Authentication and trust are inherent in DIDComm, as every message can be cryptographically verified to have originated from a specific DID. This authentication happens without centralized authorities, allowing parties to establish trust directly with one another.

Asynchronous capabilities enable communications that don't require both parties to be online simultaneously. Messages can be queued, stored, and forwarded until they reach their destination, making DIDComm suitable for a wide range of real-world scenarios where continuous connectivity isn't guaranteed.

Message routing and forwarding mechanisms allow communications to traverse complex paths, including through mediators and relays, while maintaining end-to-end security. This capability is essential for entities behind firewalls or NATs, or for preserving additional privacy by obscuring network-level metadata.

Message Structure

DIDComm Messaging messages are structured as encrypted JSON Web

Message (JWM) envelopes. This standardized format ensures interoperability while providing the necessary security properties.

A typical DIDComm message consists of:

- **Headers**: Metadata about the message, including IDs, types, timing information, and routing details
- **Body**: The actual content or payload of the message
- **Attachments**: Optional additional data, which might include structured data, documents, or media

These components are assembled into a JWM, which is then typically encrypted and/or signed according to the security requirements of the interaction. The resulting structure provides a consistent envelope format that can carry any type of content while ensuring its security and integrity.

Each message contains a unique identifier and can reference other messages through threading mechanisms, enabling complex multi-message conversations. Messages also declare their type, which connects them to specific protocols that define the expected sequence and semantics of the interaction.

Protocols

DIDComm defines not just message formats but also a framework for protocols—standardized sequences of messages that accomplish specific tasks. These protocols enable interoperability by ensuring that different implementations understand the same message sequences and semantics.

The protocol framework includes mechanisms for discovering which protocols an agent supports and for versioning protocols as they evolve. This allows for graceful upgrades and backward compatibility in the ecosystem.

Several common protocols have emerged in the SSI ecosystem:

- **Connection establishment protocols** define how two parties can establish a secure DIDComm channel
- **Credential issuance protocols** standardize how verifiable credentials are offered and issued
- **Credential presentation protocols** define how credentials can be requested and presented
- **Trust ping protocols** provide simple mechanisms to verify that a communication channel is active

By standardizing these interaction patterns, DIDComm enables different implementations to work together seamlessly, fostering an open ecosystem rather than siloed solutions.

You can find all currently published protocols in here, as these are maintained by the community, you are also encouraged to build and define your own protocols that fit your particular use case, even if you may not decide to publish them.

Example interaction

To illustrate how DIDComm works in practice, consider a typical credential issuance scenario:

1. An Issuer and Holder establish a DIDComm connection, exchanging DIDs and service endpoints
2. The Issuer sends an offer message indicating which credentials it can provide
3. The Holder responds with a request message for a specific credential
4. The Issuer creates the credential and sends it in an issuance message
5. The Holder acknowledges receipt with an acknowledgment message

Throughout this interaction, all messages are encrypted end-to-end, ensuring that even if they pass through intermediaries, the content remains private between the Issuer and Holder. The messages might travel over different transport mechanisms—perhaps starting with an HTTPS connection but switching to Bluetooth for later messages if the parties come into proximity.

In mediated scenarios, the flow becomes more complex but even more powerful. A Holder might receive messages through a mediator service that provides consistent message delivery even when the Holder's device is offline. The mediator receives encrypted messages, stores them until the Holder connects, and then forwards them without ever being able to read the encrypted contents.

Benefits

DIDComm delivers several crucial benefits that advance the core principles of Self-Sovereign Identity:

Privacy preservation stands at the forefront of DIDComm's design. By encrypting message contents and enabling pseudonymous interactions through pairwise DIDs, DIDComm allows individuals to control exactly what information they share and with whom. The transport-agnostic nature also helps prevent correlation through network metadata.

Decentralized communication frees identity interactions from dependence on centralized messaging providers or identity hubs. Parties can communicate directly or through mediators of their choosing, avoiding vendor lock-in and single points of failure.

Interoperability between different SSI implementations ensures that the ecosystem remains open and competitive. A Holder using one wallet implementation can interact seamlessly with verifiers and issuers using entirely different software stacks, as long as they all support the DIDComm protocols.

User control extends beyond just identity data to encompass the communication channels themselves. Individuals can choose how, when, and where they receive communications related to their digital identity, reinforcing the self-sovereign nature of the system.

DIDComm in Identus

Identus has adopted DIDComm v2 as its standard communication protocol, implementing it throughout the framework to enable secure, private messaging between DIDs. This implementation standardizes many important interactions, including the critical communication between mediators and their peers.

The integration with mediators is particularly important in the Identus architecture. Mediators serve as message handlers that can receive, store, and forward DIDComm messages, enabling asynchronous communication even when recipients are temporarily offline. This capability is essential for practical, real-world identity solutions that must function reliably across diverse connectivity scenarios.

At the moment of writing this chapter Identus supports DIDComm over HTTP endpoints by polling, which is by far the most tested and stable implementation, but also via WebSockets enabled as a feature flag in the Cloud Agent.

For details on how messages are sent and received in Identus via mediators, see Chapter .

Example Project

Swift SDK

TODO: Example code for send/receive DIDCOMM messages on Swift SDK

TypeScript SDK

TODO: Example code for send/receive DIDCOMM messages on TypeScript SDK

Connections

Overview

Now that we have a better understanding of Wallets and DIDs, it's time to embark on our first interaction. In this chapter we are going to explore conceptually what a Connection means in SSI, take a deep dive into DID Peers, explain how they work and why they are needed for secure connections, dissect Out of Band invites and finally hands on example code to achieve connecting edge client to an agent.

Before we move forward we highly recommend to at least read the basic Connection tutorial on the official Identus documentation.

Connections in Self-Sovereign Identity

Connections are fundamental to establishing *trusted interactions* between peers. They enable secure and verifiable communication, allowing entities to exchange credentials and proofs in a decentralized manner. This relationship is established using a specific decentralized identifier standard (**Peer DID**) and is governed by a protocol (**DIDComm**) that ensure the authenticity, integrity, and privacy of the interactions between the connected parties.

There are three roles in an SSI connection:

- **Inviter**: The entity that initiates the connection by sending an invitation.
- **Invitee**: The entity that receives the invitation and responds with a connection request.
- **Mediator**: An intermediary that facilitates message delivery between entities, especially when one or both parties may not always be online.

> **Note**
>
> We will cover mediators in detail later on. For now what you need to understand is that they are used as a service to relay messages between peers, they will store messages and deliver them whenever a peer comes back online, connects to the mediator and fetches their messages.

PeerDIDs

They are a special kind of decentralized identifier with some unique properties that allow them to be perfect for use in order to establish private and secure communications between peers.

DID Documents such as PrismDIDs are meant to be publicly available and resolvable by arbitrary parties, therefor storing them in a VDR such as Cardano blockchain is an excellent way to achieve this requirement in a reliable way.

However, when Alice and Bob want to interact with each other, only two parties care about the details of that connection: Alice and Bob. Instead of arbitrary parties needing to resolve their DIDs, only Alice and Bob do. Thus, PeerDIDs essentially describe a key-pair to encrypt and sign data to and from Alice and Bob, routed trough their preferred mediators, e.g. When Alice accepts an invite from Bob and they engage the connection protocol, Alice generates a PeerDID that allows her to encrypt and sign data routed trough Bob's mediator (mediator Y) that only Bob can decrypt, and vice versa, Bob will generate a PeerDID that allows him to encrypt and sign data routed trough Alice's preferred mediator (mediator X) that only Alice can decrypt.

The key benefits of PeerDIDs are:

1. Decentralized by nature.
2. No transaction cost on blockchain.
3. Private (only the concerned parties know about them).
4. Reusable without any reliance on the internet, with no degradation of trust. (adheres to the principles of local-first and offline-first)

Lets resolve a PeerDID, we call resolve to the unpacking and parsing of a DID in order to read its content and use the DID for the interactions that we need to achieve. For this example we will resolve a PeerDID from Atala's mediator sandbox.

```
curl https://sandbox-mediator.atalaprism.io/did

did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6Mkhh1e5CEYYq6JBUcTZ6Cp2ranCW
```

We can see that the mediator returned a PeerDID when we send a `GET` request to the `/did` endpoint. In order to resolve this DID we can use an Universal Resolver website for this example:

```
curl https://dev.uniresolver.io/1.0/identifiers/did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsH
```

```
{
  "@context": "https://w3id.org/did-resolution/v1",
  "didDocument": {
    "@context": [
      "https://www.w3.org/ns/did/v1",
      "https://w3id.org/security/multikey/v1",
```

```
      {
        "@base": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7
      }
    ],
    "id": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6Mk
    "verificationMethod": [
      {
        "id": "#key-2",
        "type": "Multikey",
        "controller": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvM
        "publicKeyMultibase": "z6Mkhh1e5CEYYq6JBUcTZ6Cp2ranCWRrv7Yax3Le4N59R6
      },
      {
        "id": "#key-1",
        "type": "Multikey",
        "controller": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvM
        "publicKeyMultibase": "z6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm
      }
    ],
    "keyAgreement": [
      "#key-1"
    ],
    "authentication": [
      "#key-2"
    ],
    "assertionMethod": [
      "#key-2"
    ],
    "service": [
      {
        "serviceEndpoint": {
          "uri": "https://sandbox-mediator.atalaprism.io",
          "accept": [
```

```json
            "didcomm/v2"
          ]
        },
        "type": "DIDCommMessaging",
        "id": "#service"
      },
      {
        "serviceEndpoint": {
          "uri": "wss://sandbox-mediator.atalaprism.io/ws",
          "accept": [
            "didcomm/v2"
          ]
        },
        "type": "DIDCommMessaging",
        "id": "#service-1"
      }
    ]
  },
  "didResolutionMetadata": {
    "contentType": "application/did+ld+json",
    "pattern": "^(did:peer:.+)$",
    "driverUrl": "http://uni-resolver-driver-did-uport:8081/1.0/identifiers/",
    "duration": 4,
    "driverDuration": 4,
    "did": {
      "didString": "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6Mkhh1e5C
      "methodSpecificId": "2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6Mkhh1e5CEY
      "method": "peer"
    }
  },
  "didDocumentMetadata": {}
}
```

This is what a PeerDID looks like when resolved, please bear in mind that the JSON-LD context for `did-resolution` and extra `didResolutionMetadata` entry are added by the resolver and the actual isolated PeerDID is only what we see inside the `didDocument` payload.

In simple terms, a PeerDID is essentially a JSON payload that contains a set of keys and an optional service endpoints, because this is a mediator PeerDID, it contains service endpoints for `DIDCommMessaging`, in this particular case, you can see it contains two of them, one over regular `https` and other trough `websockets`.

To go deeper in your understanding of PeerDIDs please refer to the full Peer DID Method Specification. In the Hyperledger Identus ecosystem, only PeerDIDs method 2 are supported at the time of this writing.

Out of Band invites

Out of Band (OOB) invites are the entry point for some protocols to take place, they usually are encoded in either a JSON payload or a URL and are distributed "out of band", usually over QR codes, but could be distributed over any medium (Bluetooth, NFC, etc). They gather all required information for one peer to start interacting with another and you can think of them as a way to advertise "coordinates" for anyone that would like to establish an interaction to the inviter.

Following the same example as before, we can see that the sandbox mediator also delivers an OOB invite in a URL form.

```
https://sandbox-mediator.atalaprism.io?_oob=eyJpZCI6ImExNTY4YzEyLTBjZGMtNDY0N
```

If we decode the value of the `_oob` query variable from `base64` we get the `json` payload

```json
{
  "id" : "a1568c12-0cdc-4647-9dc6-a5aada6f8247",
  "type" : "https://didcomm.org/out-of-band/2.0/invitation",
  "from" : "did:peer:2.Ez6LSghwSE437wnDE1pt3X6hVDUQzSjsHzinpX3XFvMjRAm7y.Vz6Mkhh1e5CEYYq6JBU
  "body" : {
    "goal_code" : "request-mediate",
    "goal" : "RequestMediate",
    "accept" : [
      "didcomm/v2"
    ]
  },
  "typ" : "application/didcomm-plain+json"
}
```

As you can see, an Out of Band invite is really just a way to package a PeerDID and signaling that it can be used for a particular interaction. In this case, for a `RequestMediate` goal over `DIDComm`.

As a refresher from what we covered, a PeerDID is a way to package a set of keys and optional service endpoints, and so, *because this an OOB invite from a mediator*, this invite has everything you need (a PeerDID and set of service endpoints) to establish this service as your mediator.

Connecting two peers

Now, lets issue another kind of Out of Band invite, one from the cloud agent in order to connect.

The Cloud Agent can generate Out of Band invites, this invite then can be parsed by another peer (say another Cloud Agent or Edge Client) and use it to establish a connection, the end result should be a DID Peer on both sides that allow them to send messages to each other over `DIDComm`.

So, our first step is to generate the invite:

```
curl --location 'http://127.0.0.1:8080/cloud-agent/connections' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--data '{"label": "test"}'
```

> **Note**
>
> The only parameter we can change when generating an invite is the `label`, this is optional and it is a simple string that you can use to identify the connection later, a good idea would be to use a `uuid` that you generate and manage on your systems, or could be an alias or the reason for the connection, this is all contextual to the interaction and use case so in our case we will go with "test".

The Cloud Agent will respond with a payload that should look like this:


```
{
    "connectionId": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "thid": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "label": "test",
    "role": "Inviter",
    "state": "InvitationGenerated",
    "invitation": {
        "id": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
        "type": "https://didcomm.org/out-of-band/2.0/invitation",
        "from": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cp
        "invitationUrl": "https://my.domain.com/path?_oob=eyJpZCI6ImZiMzZlZG
    },
    "createdAt": "2025-01-04T12:37:37.059649293Z",
    "metaRetries": 5,
    "self": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
```

```
    "kind": "Connection"
}
```

Once the connection invite is created you can fetch it's details by a `GET`request passing the `connectionId`:

```
curl --location 'http://127.0.0.1:8080/cloud-agent/connections/fb36eddd-d51e-42cf-a6fe-e76d2
--header 'Accept: application/json' \
```

You should get back the same payload as when it was created unless something changed, like the `state`:

```
{
    "connectionId": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "thid": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "label": "test",
    "role": "Inviter",
    "state": "InvitationGenerated",
    "invitation": {
        "id": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
        "type": "https://didcomm.org/out-of-band/2.0/invitation",
        "from": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7H7SNA6
        "invitationUrl": "https://my.domain.com/path?_oob=eyJpZCI6ImZiMzZlZGRkLWQ1MWUtNDJjZi
    },
    "createdAt": "2025-01-04T12:37:37.059649Z",
    "metaRetries": 5,
    "self": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
    "kind": "Connection"
}
```

Lets dig a bit deeper on what this payload represents.

This invite payload contains some important metadata:

- **connectionId**: The unique identifier of the connection resource, used to fetch the connection details.
- **thid**: The unique identifier of the *thread* this connection record belongs to. The value will identical on both sides of the connection (inviter and invitee).
- **label**: A human readable alias for the connection.
- **role**: The Cloud Agent role on this connection, either `Inviter` or `Invitee`.
- **state**: The current status of this connection, note this is contextual to the Cloud Agent role, so as `Inviter` the states could be: `InvitationGenerated`, `ConnectionRequestReceived`, `ConnectionResponsePending`, `ConnectionResponseSent`. But is also possible for the Cloud Agent to parse someone else's invitation, in that case the Cloud Agent will generate a connection with the `Invitee` role and the possible states for that role are: `InvitationReceived`, `ConnectionRequestPending`, `ConnectionRequestSent`, `ConnectionResponseReceived`.
- **invitation**: The `DIDComm` invitation details.
- **createdAt**: Date and time when this connection was created or received.
- **metaRetries**: The maximum background processing attempts remaining for this record.
- **self**: The reference to the connection resource.
- **kind**: The type of object returned. In this case a `Connection`.

Now lets unpack the `invitation` details.

- **id**: The unique identifier of the invitation. It should be used as parent thread ID (`pthid`) for the Connection Request message that follows.
- **type**: The DIDComm Message Type URI (MTURI) the invitation message complies with.
- **from**: The DID representing the sender to be used by recipients for future interactions.

- **invitationUrl**: The invitation message encoded as a URL.

Lets resolve the `from` DID:

```
curl https://dev.uniresolver.io/1.0/identifiers/did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaE
```

```
{
  "@context": "https://w3id.org/did-resolution/v1",
  "didDocument": {
    "@context": [
      "https://www.w3.org/ns/did/v1",
      "https://w3id.org/security/multikey/v1",
      {
        "@base": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7H7SNA
      }
    ],
    "id": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7H7SNA6ooG5sn
    "verificationMethod": [
      {
        "id": "#key-2",
        "type": "Multikey",
        "controller": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7
        "publicKeyMultibase": "z6MkpX7H7SNA6ooG5snn2MzgyoRadEZtsjNSL1x7HiiLkqyV"
      },
      {
        "id": "#key-1",
        "type": "Multikey",
        "controller": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7
        "publicKeyMultibase": "z6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj"
      }
    ],
    "keyAgreement": [
      "#key-1"
    ],
```

```
    "authentication": [
      "#key-2"
    ],
    "assertionMethod": [
      "#key-2"
    ],
    "service": [
      {
        "serviceEndpoint": {
          "uri": "http://host.docker.internal:8080/didcomm",
          "routingKeys": [],
          "accept": [
            "didcomm/v2"
          ]
        },
        "type": "DIDCommMessaging",
        "id": "#service"
      }
    ]
  },
  "didResolutionMetadata": {
    "contentType": "application/did+ld+json",
    "pattern": "^(did:peer:.+)$",
    "driverUrl": "http://uni-resolver-driver-did-uport:8081/1.0/identifiers/
    "duration": 3,
    "driverDuration": 3,
    "did": {
      "didString": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA
      "methodSpecificId": "2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7c
      "method": "peer"
    }
  },
  "didDocumentMetadata": {}
```

```
}
```

Now this looks familiar, we have the usual set of keys and it looks similar to the mediator DID but in this case we see a `serviceEndpoint` that contains accepts `didcomm/v2` and it's intended to be used for `DIDCommMessaging`. What all this means is the Cloud Agent DID is essentially advertising how it will receive `DIDComm` messages. In other words this could be translated as: "Here is an invite to connect to me, on it you will find a DID that has my public keys and a service endpoint where I receive `DIDComm` messages".

The final piece to unpack is the `invitationUrl`, this looks a little odd at first:

```
"https://my.domain.com/path?_oob=eyJpZCI6ImZiMzZlZGRkLWQ1MWUtNDJjZi1hNmZlLWU3NmQyZTYzOGI3MCI
```

The first thing that looks wrong is the domain, where does `my.domain.com` comes from? well, it comes from the Cloud Agent and it's hardcoded, you can't customize it but it really doesn't matter, what matters is the payload of the `_oob` field. The URL is not important as what we really need is inside the `base64` encoded field, lets unpack it.

```
echo 'eyJpZCI6ImZiMzZlZGRkLWQ1MWUtNDJjZi1hNmZlLWU3NmQyZTYzOGI3MCIsInR5cGUiOiJodHRwczovL2RpZC
```

```
{
  "id": "fb36eddd-d51e-42cf-a6fe-e76d2e638b70",
  "type": "https://didcomm.org/out-of-band/2.0/invitation",
  "from": "did:peer:2.Ez6LSgY6Y67mJ75YCZfZYxYEPQJZs3vaEg2Cc91vppoTA7cpj.Vz6MkpX7H7SNA6ooG5sn
  "body": {
    "accept": []
  }
}
```

And there it is, the `_oob` encoded payload contains the bare minimum to tell you it's a `DIDComm` invitation from a DID Peer.

Connectionless Credential Presentations and Issuance

In the realm of digital identity, establishing trust and exchanging verifiable credentials typically involves a series of interactions between an issuer or verfier and a holder. While many protocols need an existing connection or relationship, a connectionless flow offers a more streamlined approach for specific scenarios. This method is particularly useful when a prior relationship between the issuer or verfier and the holder has not yet been established or is not necessary for a particular interaction.

The fundamental distinction of connectionless issuance and presentation lies in its initiation. Unlike traditional flows that might require a formal connection setup (e.g., exchanging DIDs and establishing a DIDComm channel) *before* a credential offer can be made, the connectionless flow leverages an Out-of-Band (OOB) invitation to kickstart the process directly.

When an issuer or verifier intends to offer or request proof this way, they generate an OOB invitation. This invitation is essentially a self-contained message or a reference (like a URL or QR code) that the holder can receive through various means (QR code, email, a website, etc). Upon parsing this OOB invitation, the holder's agent can immediately understand the intent to offer or proof a credential.

The key here is that the OOB invitation itself contains enough information, or points to it, for the holder's agent to proceed with requesting the credential offer from the issuer or presenting proof to a verifier. This bypasses the need for a separate, preliminary connection handshake. The issuer or verifier, upon receiving a request derived from this OOB invitation, can then proceed to send the actual credential offer or proof request.

From the holder's acceptance of the offer onwards, the subsequent steps closely mirror those in a standard issuance and proof request protocols. The primary efficiency and distinction of the connectionless approach are concentrated at the beginning of the interaction, enabling a quicker, more direct path when a persistent connection is not a prerequisite. This makes it ideal for scenarios like anonymous attestations, one-time verifications, or public credential offerings where the overhead of establishing and managing connections for every holder is impractical.

Example Project

Cloud Agent

TODO: Instructions on how to create an invite on Cloud Agent.

Swift SDK

TODO: Example code to parse invites on Swift SDK

TypeScript SDK

TODO: Example code to parse invites on TypeScript SDK

Verifiable Credentials

Overview

Think of a Verifiable Credential (VC) as a signed statement that can move with the person or organization it describes. An airline can issue a boarding pass to a traveler. A university can issue a degree credential to a graduate. A government agency can issue a permit to a company. In each case, the holder keeps the credential and later presents it to a verifier that needs proof of a claim.

The W3C Verifiable Credentials Data Model 2.0 defines the main roles as issuer, holder, verifier, and subject. The issuer creates the credential. The holder stores it and presents it. The verifier checks the presentation. The subject is the person, organization, thing, or account that the claims describe. The holder and subject sometimes match. In other cases, a parent may hold a child credential, or a company administrator may hold credentials about the company.

A VC contains four kinds of information:

- Claims, such as name, ticket number, membership level, age range, or license category.
- Issuer information, so a verifier can decide who made the statement.
- Proof material, so a verifier can check integrity and authorship.
- Metadata, such as issuance date, expiration date, schema, credential type, and status.

The proof lets a verifier detect tampering. It does not decide whether the verifier should accept the credential for a given business purpose. That decision depends on policy. A boarding gate may accept only credentials issued by the airline for the current flight. A verifier may reject an otherwise valid credential if the issuer is not trusted, the credential has expired, the schema is not accepted, or the credential has been revoked.

The Identus Cloud Agent README describes Cloud Agent as a W3C and Aries-based agent that can issue, hold, and verify credentials over DIDComm v2. Identus supports multiple credential formats through a REST API and webhook-driven agent workflows.

Formats

VCs share a common purpose, but they do not all use the same envelope, proof format, privacy model, or verification procedure. Identus Cloud Agent documents three credential formats for issuance through its APIs: JWT-VC, SD-JWT-VC, and AnonCreds. In Cloud Agent payloads these appear as `credentialFormat` values such as `JWT`, `SDJWT`, and `AnonCreds`.

JWT Verifiable Credentials

JWT-VC packages credential claims in a JSON Web Token. Teams often choose it when their verifiers already rely on JOSE tooling and DID-based issuer verification. The verifier decodes the JWT, validates the signature against the issuer's public key resolved from the issuer DID, and reads the claims.

JWT-VC has a privacy tradeoff. The holder usually presents the credential as a whole, so the verifier sees all claims in the credential. That is fine for a boarding gate that needs the full ticket details. It is a poor fit when the holder should reveal only one or two fields.

The Identus DIDComm issuance guide uses `jwtVcPropertiesV1` for JWT credential offers and notes that this payload shape is aligned with the W3C VC Data Model 1.1 profile used by the agent.

SD-JWT Verifiable Credentials

SD-JWT-VC builds on Selective Disclosure JWT. It lets a holder disclose selected claims from a credential instead of revealing the whole claim set. A traveler proving lounge access may need to disclose membership tier and expiration date, but not date of birth or full customer profile.

The IETF SD-JWT VC draft defines data formats and processing rules for JSON credentials with selective disclosure. At a high level, the issuer signs a structure that commits to the claim values. The holder later presents only the disclosures needed for the verifier's request, and the verifier checks those disclosures against the signed commitments.

Identus uses `sdJwtVcPropertiesV1` for this format and requires Ed25519 issuer and holder keys for SD-JWT issuance. Identus Present Proof documentation calls out one presentation detail: if the SD-JWT credential has no `cnf` key, the holder cannot create and sign a presentation bound to the verifier's challenge and domain. With `cnf`, the holder can prove control of the bound key during presentation.

AnonCreds

AnonCreds is a zero-knowledge proof credential format. It comes from the Hyperledger Indy ecosystem and is now maintained under Hyperledger AnonCreds. Holders use it for privacy-preserving proofs, such as proving predicates over attributes without disclosing raw values.

AnonCreds requires a credential definition. That artifact references a schema and contains issuer cryptographic material used for proof generation. In Identus, AnonCreds claims use a flat string-to-string shape, and the issuer references the credential definition with `credentialDefinitionId` inside `anoncredsVcPropertiesV1`.

OID4VCI

OpenID for Verifiable Credential Issuance (OpenID4VCI) is an issuance protocol, not a credential format. It extends OAuth 2.0-style flows so a wallet can obtain credentials from an issuer through a Credential Endpoint. The OpenID Foundation approved OpenID4VCI 1.0 as a Final Specification on 16 September 2025, and the specification defines the issuer metadata, credential configurations, authorization flow, and credential request flow.

The Identus OID4VCI guide describes Cloud Agent acting as a Credential Issuer server and integrating with an Authorization Server that follows the expected contract. Identus provides an example flow with Keycloak and authorization code issuance.

W3C VC 2.0 context

Early VC 1.1 deployments no longer describe the full standards picture. W3C announced the Verifiable Credentials 2.0 family as Recommendations on 15 May 2025. The VC JOSE and COSE Recommendation defines ways to secure VC Data Model 2.0 credentials and presentations with JOSE, SD-JWT, and COSE.

Identus documentation still names the concrete formats and payloads supported by Cloud Agent, so implementation work should follow the Cloud Agent docs first and treat the W3C VC 2.0 documents as the direction of the wider standards family.

Schemas

A credential schema describes the shape of credential claims. It answers questions such as: Which fields are expected? Which fields are required? What data type should each field use? A schema does not decide whether a claim is true. It gives issuers, holders, and verifiers a shared structure for reading the credential.

Schema specifications

Identus uses two schema families across its credential formats.

JWT and SD-JWT credentials use JSON Schema. The official Identus schema guide requires the schema body to set `$schema` to `https://json-schema.org/draft/2020-12/schema`, which matches the current draft described by the JSON Schema project.

AnonCreds credentials use AnonCreds schemas paired with credential definitions. The credential definition references a schema by ID and must exist before the issuer can create an AnonCreds credential offer.

Example schema

An airline ticket credential might use a JSON Schema like this:

```
{
  "$id": "https://example.com/schemas/boarding-pass-1.0",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Boarding pass",
  "type": "object",
  "properties": {
    "passengerName": { "type": "string" },
    "flightNumber": { "type": "string" },
```

```
    "departureAirport": { "type": "string" },
    "arrivalAirport": { "type": "string" },
    "departureDateTime": { "type": "string", "format": "date-time" },
    "seatNumber": { "type": "string" },
    "boardingGroup": { "type": "string" },
    "ticketClass": { "type": "string" }
  },
  "required": [
    "passengerName",
    "flightNumber",
    "departureAirport",
    "arrivalAirport",
    "departureDateTime",
    "seatNumber"
  ],
  "additionalProperties": false
}
```

Creating schemas in Identus

Identus Cloud Agent has a schema registry for creating and resolving credential schemas. The official schema guide documents two creation endpoints:

```
POST /cloud-agent/schema-registry/schemas
POST /cloud-agent/schema-registry/schemas/did-url
```

Use the HTTP endpoint when verifiers should resolve the schema through an HTTP URL. Use the DID URL endpoint when verifiers should resolve the schema through a DID URL. These are separate resolution modes, so keep the expected verifier path in mind before issuing credentials against the schema.

Each schema record includes metadata around the schema definition. The `name` gives the schema a readable label, `version` identifies the version, `author` names the DID that created it, and `tags` help filter schema records. The `schema` field contains the JSON Schema definition. Cloud Agent treats the combination of `author`, schema `id`, and `version` as unique.

The `author` field must be the short-form PRISM DID created by the same Cloud Agent. The DID does not need to be published before schema creation. During setup, an issuer can create a DID, create a schema, and publish the DID later when the credential flow needs public resolution.

Versioning

Updating a schema means creating a new schema record with the changed JSON Schema and a higher version. Do not mutate the meaning of a schema version after credentials have been issued against it. Old credentials remain easier to verify when their original schema still resolves to the same structure.

For breaking changes, such as removing required fields or changing field types, use a new major version. For additive changes, such as adding an optional field, use a minor version. This keeps issuer and verifier policy readable when multiple credential versions exist at the same time.

Issuing

Issuance turns an issuer's claim data into a credential held by a wallet or agent. In Identus, issuance can happen through a DIDComm connection, through a connectionless invitation, or through OpenID4VCI.

The Identus DID guides matter before issuance starts. The Cloud Agent registrar can create a PRISM DID offline. The DID document template supports verification methods for purposes such as `authentication` and `assertionMethod`. A long-form PRISM DID carries the create operation in the DID itself. A short-form PRISM DID needs publication before other parties can resolve it from the ledger. The Cloud Agent DID publication guide documents the publication flow and its status changes from `CREATED` to `PUBLICATION_PENDING` to `PUBLISHED`.

Prerequisites

For DIDComm issuance, the Identus Issue Credential Protocol 3.0 guide describes the format-specific prerequisites.

JWT issuance requires a connection between issuer and holder agents, a published issuer PRISM DID with `assertionMethod`, a credential schema, and a holder PRISM DID with `authentication`.

SD-JWT issuance has the same shape, but the issuer and holder PRISM DIDs must use Ed25519 keys for the relevant verification methods.

AnonCreds issuance requires a connection and an AnonCreds credential definition.

Credential states

The issuer-side states are:

```
OfferPending -> OfferSent -> RequestReceived -> CredentialPending -> Credenti
```

The holder-side states are:

```
OfferReceived -> RequestPending -> RequestSent -> CredentialReceived
```

These states are useful when a wallet or controller needs to resume a flow, show progress to a user, or decide whether a manual issuer action is still required.

Issuer flow

An issuer starts by preparing four pieces of data: the issuer DID, the credential schema or credential definition, the claims, and the credential format.

For a boarding pass, the airline selects its issuer DID, chooses the boarding pass schema, and prepares claims such as passenger name, flight number, departure date, seat, and boarding group. The format choice depends on verifier needs. A gate scanner may only need a JWT-VC. Use SD-JWT-VC when the traveler should reveal only part of the credential. Use AnonCreds when the verifier needs zero-knowledge proofs or predicate proofs.

The issuer creates a credential offer through Cloud Agent:

```
POST /cloud-agent/issue-credentials/credential-offers
```

JWT offers use `jwtVcPropertiesV1`:

```json
{
  "connectionId": "holder-connection-id",
  "credentialFormat": "JWT",
  "jwtVcPropertiesV1": {
    "claims": {
      "passengerName": "Alice Wonderland",
      "flightNumber": "ID123",
      "departureAirport": "SCL",
      "arrivalAirport": "EZE",
      "seatNumber": "12A"
    },
    "issuingDID": "did:prism:issuer",
    "credentialSchema": {
      "id": "http://localhost:8080/cloud-agent/schema-registry/schemas/...",
      "type": "JsonSchemaValidator2018"
```

```
    }
  }
}
```

SD-JWT offers use `sdJwtVcPropertiesV1`:

```json
{
  "connectionId": "holder-connection-id",
  "credentialFormat": "SDJWT",
  "sdJwtVcPropertiesV1": {
    "claims": {
      "passengerName": "Alice Wonderland",
      "flightNumber": "ID123",
      "boardingGroup": "A",
      "exp": 1883000000
    },
    "issuingDID": "did:prism:issuer",
    "credentialSchema": {
      "id": "http://localhost:8080/cloud-agent/schema-registry/schemas/...",
      "type": "JsonSchemaValidator2018"
    }
  }
}
```

AnonCreds offers use `anoncredsVcPropertiesV1` and a `credentialDefinitionId`:

```json
{
  "connectionId": "holder-connection-id",
  "credentialFormat": "AnonCreds",
  "anoncredsVcPropertiesV1": {
    "claims": {
      "passengerName": "Alice Wonderland",
```

```
      "flightNumber": "ID123",
      "seatNumber": "12A"
    },
    "issuingDID": "did:prism:issuer",
    "credentialDefinitionId": "credential-definition-id"
  }
}
```

After creating the offer, the issuer watches the issue credential record state. If the holder accepts the offer and the issuer uses automatic issuance, Cloud Agent issues the credential when the holder's request arrives. If automatic issuance is disabled, the issuer sends the credential from the matching record:

```
POST /cloud-agent/issue-credentials/records/{recordId}/issue-credential
```

Holder flow

The holder receives a credential offer, reviews it, and accepts it. In a user-facing wallet, that review step should show the issuer, credential type, requested format, claim preview, and any privacy-relevant details. The holder should know what they are accepting before the wallet stores it.

The holder can retrieve issue credential records:

```
GET /cloud-agent/issue-credentials/records
```

To accept a JWT or SD-JWT offer, the holder provides a PRISM DID as `subjectId`:

```json
{
  "subjectId": "did:prism:holder"
}
```

For SD-JWT credentials with key binding, the holder can include `keyId`:

```json
{
  "subjectId": "did:prism:holder",
  "keyId": "key-1"
}
```

AnonCreds offers do not use `subjectId` in the same way. The holder accepts the offer with an empty body:

```json
{}
```

After acceptance, the holder receives the issued credential over DIDComm. Identus documents holder states such as `OfferReceived`, `RequestPending`, `RequestSent`, and `CredentialReceived`. The wallet stores the credential and later uses it in a presentation flow.

Connectionless issuance

Connectionless issuance helps when the issuer and holder do not already have a DIDComm connection. The Identus connectionless issuance guide documents an out-of-band invitation endpoint:

```
POST /cloud-agent/issue-credentials/credential-offers/invitation
```

The issuer creates an invitation and shares the returned invitation code, for example as a QR code or link. The holder accepts the invitation through a wallet or SDK. The issuer responds with a credential offer, and the normal holder acceptance and issuer issuance steps follow.

A conference check-in desk could display a QR code that starts issuance of an attendee credential. The attendee scans it with a wallet, accepts the offer, and receives the credential without first creating a standing relationship with the issuer.

OpenID4VCI issuance

OID4VCI fits issuers that already use OAuth-based user journeys. DIDComm issuance fits wallets and issuers that already participate in agent-to-agent protocols. A production deployment may support both paths, with policy deciding which one applies to each credential type.

In the Identus OID4VCI model, Cloud Agent acts as the Credential Issuer server. The wallet follows the authorization flow, obtains an access token from the Authorization Server, and uses that token at the Credential Endpoint. The exact issuer policy lives in the issuer and authorization server configuration, not in the wallet.

Presenting and verification

Issuance gives the holder a credential. Presentation is how the holder proves something with it.

In Identus, the Present Proof Protocol 3.0 guide describes verifier, holder, and prover flows over DIDComm. A verifier creates a proof request:

```
POST /cloud-agent/present-proof/presentations
```

The request can include a `domain` and `challenge`. These values bind the presentation to the verifier and session, which reduces replay risk. The holder reviews the request, selects credentials, and accepts it:

```
PATCH /cloud-agent/present-proof/presentations/{id}
```

The exact payload depends on the credential format. JWT presentations use a `proofId`. SD-JWT presentations disclose selected claims and use `credentialFormat: SDJWT`. AnonCreds presentations use an AnonCreds presentation request.

After verification, Identus can reach `PresentationVerified`. The verifier can then accept the presentation, which moves the record to `PresentationAccepted`. This split matters. Verification checks cryptographic validity and protocol rules. Acceptance records a business decision by the verifier.

In the airline example, a valid ticket credential may still fail business policy. A traveler can hold a legitimate ticket and still lack a required visa, arrive after the gate closes, or fail a security check. VC verification tells the verifier whether the credential is authentic, intact, and acceptable under protocol rules. The verifier still decides whether to accept it for the trip.

Revoking

Revocation lets an issuer mark a credential as no longer valid before its natural expiration. An airline may revoke a boarding pass after a booking is canceled. A university may revoke a credential issued by mistake. A company may revoke an employee credential after employment ends.

The Identus revocation guide documents revocation for JWT credentials using Verifiable Credentials Status List v2021. A revocable credential contains a `credentialStatus` object with fields such as `type`, `statusPurpose`, `statusListIndex`, and `statusListCredential`.

Revocation mechanism

A revocable credential contains status data like this:

```json
{
  "credentialStatus": {
    "id": "http://localhost:8080/cloud-agent/credential-status/[UUID]#[INDEX]",
    "type": "StatusList2021Entry",
    "statusPurpose": "revocation",
    "statusListIndex": "94567",
    "statusListCredential": "http://localhost:8080/cloud-agent/credential-status/[UUID]"
  }
}
```

The `statusListCredential` URL points to a Status List Credential. That credential contains an encoded bitstring where each bit position corresponds to a credential. The `statusListIndex` identifies the credential's position in that bitstring. When the issuer revokes a credential, the issuer flips the corresponding bit from 0 to 1.

Revoking a credential

Only the issuer of a credential can revoke it. Identus exposes the revocation operation as:

```
PATCH /cloud-agent/revoke-credential/{credential_id}
```

The issuer can find the credential ID by listing issued credential records, then calling the revocation endpoint for the target credential. Once revoked, a later Present Proof verification fails if the holder presents that credential. This keeps revocation in the verification path instead of relying on the holder to remove old credentials from storage.

Checking revocation status

When a verifier receives a presentation containing a credential with `credentialStatus`, the verifier checks the status list before accepting the credential.

First, the verifier retrieves the Status List Credential from `credentialStatus.statusListCre` Then it verifies the proof embedded in that status credential. Identus documents two supported proof types for status list credentials: `DataIntegrityProof` with `eddsa-jcs-2022` for Ed25519 keys, and `EcdsaSecp256k1Signature2019` for secp256k1 keys.

After proof verification, the verifier decodes `credentialSubject.encodedList` and checks the bit at `statusListIndex`. If the bit is set to 1, the credential has been revoked. If the bit is 0, the status list has not revoked that credential.

This status-list model has a privacy benefit: the verifier retrieves the whole list instead of asking the issuer about a specific credential. The issuer does not learn which credential the verifier checked.

W3C published Bitstring Status List v1.0 as part of the VC 2.0 Recommendation family on 15 May 2025. The Identus documentation cited above still names Status List v2021 for JWT credential revocation, so implementers should follow the current Identus guide for Cloud Agent behavior and track future Identus releases for any status-list migration.

Verification

Presenting proof

Verification in Identus starts when a verifier asks a holder to present evidence from one or more credentials. For DIDComm flows, Identus Cloud Agent uses the Present Proof protocol. The verifier Cloud Agent sends a presentation request over an existing DIDComm connection, the holder Cloud Agent returns a presentation, and the verifier Cloud Agent checks the received proof material.

The Identus Present Proof guide describes the DIDComm API flow:

1. The verifier controller calls `POST /present-proof/presentations` on its Cloud Agent. The request identifies the DIDComm connection and describes the proof the verifier wants.
2. The holder controller reads pending presentation records from `GET /present-proof/presentations`.
3. The holder controller accepts a request with `PATCH /present-proof/presentations/{id}` using `action: "request-accept"`. For JWT and SD-JWT credentials it supplies credential record IDs through `proofId`. For AnonCreds it supplies the credential proof mapping under `anoncredPresentationRequest`.
4. The holder Cloud Agent generates the presentation and sends it to the verifier Cloud Agent.
5. The verifier Cloud Agent verifies the presentation. A successful verifier-side record moves through `PresentationReceived` and `PresentationVerified`.

6. The verifier controller accepts the verified presentation with `PATCH /present-proof/presentations/{id}` using `action: "presentation-accept"`, or rejects it in application logic.

The exact request shape depends on the credential format. JWT and SD-JWT presentation requests can include `options.challenge` and `options.domain` so the holder signs material bound to the verifier's request. AnonCreds presentation requests use an `anoncredPresentationRequest` object with a nonce, requested attributes, requested predicates, restrictions, and optional non-revocation intervals.

The DIDComm Present Proof 3.0 protocol is format-neutral. It carries a presentation request and a presentation, but the credential format decides which cryptographic checks run.

Verification and acceptance

Cryptographic verification and verifier acceptance are separate decisions.

The verifier Cloud Agent can check whether the presentation is well formed for its format, bound to the verifier request, and backed by the expected cryptographic material. For JWT-VC, the verifier checks JWT signatures and DID-resolved keys. For SD-JWT-VC, the verifier checks the issuer signature, disclosed claim digests, and holder key binding when the credential carries a `cnf` key. For AnonCreds, the verifier checks equality proofs, predicate proofs, aggregate proofs, and any non-revocation proofs requested by the verifier.

The verifier controller still decides whether the proof should be accepted for the business action. The W3C Verifiable Credentials Data Model 2.0 separates verification from validation: verification checks the securing mechanism, and validation applies business rules to a specific use of the credential.

After Cloud Agent reports a verified presentation, the verifier controller should check:

1. The verifier trusts the issuer DID for the credential type and schema.
2. The credential uses the schema or credential definition named in the verifier's request.
3. The disclosed claims satisfy the verifier's policy.
4. The credential validity period and any status or revocation data fit the verifier's time of interest.
5. The verifier accepts the holder or presenter as authorized for this use case.

A valid proof can still fail acceptance. A venue verifier can receive a cryptographically valid age credential, reject it if the issuer is outside its trusted issuer list, and reject it if the revealed claim does not meet the venue's age rule.

Direct credential verification

Cloud Agent exposes a direct verification endpoint at `POST /verification/credential`. Use this endpoint when verifier software has an encoded JWT credential and needs explicit check results outside a DIDComm present-proof exchange.

The endpoint accepts a list of encoded credentials and verification checks. The current service implementation includes checks such as signature verification, algorithm verification, issuer identification, expiration, not-before, audience, schema validation, subject schema validation, and semantic parsing of claims.

The direct verification endpoint does not replace a presentation request. It verifies credential data supplied to the endpoint. It does not prove that the current holder controls the credential subject's key for a verifier challenge, and it does not decide whether an issuer or claim should be accepted by the application.

Verification policies

Identus uses the term `verification policy` for stored verifier rules. The Cloud Agent source exposes `/verification/policies` endpoints to create, update, fetch, list, and delete those policies. The current policy model contains a `CredentialSchemaAndTrustedIssuersConstraint` with:

1. `schemaId`, the schema the verifier expects.
2. `trustedIssuers`, the issuer DIDs the verifier accepts for that schema.

The endpoint description in the Cloud Agent source says these policies apply to W3C Verifiable Credentials in JWT format and currently use `schemaId` and `trustedIssuers` constraints.

A verifier controller can use a verification policy before sending a presentation request, after receiving a verified presentation, or both. Before the request, the policy can help the verifier ask for the schema it accepts. After verification, the policy can help the verifier reject a credential from an untrusted issuer. The Cloud Agent's cryptographic verification result should feed into this policy decision; it should not be treated as the whole decision.

If a deployment uses an external trust registry, keep the registry lookup separate from proof verification. The verifier or its controller queries the registry, resolves the trusted issuer set for the requested schema, and passes that result into its policy decision.

Selective disclosure

Selective disclosure lets a holder reveal only the claims needed for a verifier request. Identus supports selective-disclosure presentations through SD-JWT and AnonCreds, but the two formats use different mechanisms.

Plain JWT-VC presentation exposes the credential payload needed for verifier checks. The format works for use cases where the verifier needs the full credential, such as a boarding gate checking a complete boarding pass. It fits poorly when the verifier needs one attribute or one predicate.

SD-JWT uses salted hashes. The issuer signs a JWT payload that contains digests for selectively disclosable claims. During presentation, the holder sends the issuer-signed SD-JWT plus the disclosures selected for that verifier. The verifier hashes each disclosed value with its salt and checks that the digest appears in the signed payload. SD-JWT gives selective disclosure for JSON claims; it is not a zero-knowledge predicate proof system.

Identus adds one key-binding detail for SD-JWT presentations. If the holder accepts an SD-JWT credential offer with a `keyId`, the issued credential can include a `cnf` key binding. A later holder presentation can then sign the verifier's `challenge` and `domain`. If the SD-JWT credential has no `cnf` key, the Identus Present Proof guide says the holder cannot create a presentation that signs the verifier's `challenge` and `domain`.

AnonCreds uses zero-knowledge proofs. The verifier's presentation request can ask for revealed attributes, predicates such as `age >= 18`, restrictions such as `schema_id` or `cred_def_id`, and non-revocation intervals. The holder selects credentials that satisfy the request and creates a proof. The verifier resolves the required schemas, credential definitions, and revocation data, then verifies the presentation.

AnonCreds lets the holder prove a predicate without revealing the raw attribute. For age verification, a government issuer can issue a credential containing a date-derived age attribute. An age-restricted venue can request an AnonCreds predicate for `age >= 21`. The holder presents a proof that satisfies the predicate, and the verifier checks the proof, issuer restriction, credential definition, and revocation interval without receiving the holder's full date of birth.

Selective disclosure does not remove verifier policy. The verifier still checks issuer trust, schema fit, credential status, request binding, and local acceptance rules for the disclosed claims or predicates.

Section IV - Deploy

Installation - Production Environment

Overview

A production environment setup requires connecting Hyperledger Identus to the Cardano blockchain as the Verifiable Data Registry (VDR). This is achieved through the `prism-node` component, which abstracts the VDR operations for publishing, resolving, updating, and deactivating Decentralized Identifiers (DIDs).

According to the official documentation:

> The PRISM Node generates a transaction with information about the DID operation and verifies and validates the DID operation before publishing it to the blockchain. Once the transaction gets confirmed on the blockchain, the PRISM Node updates its internal state to reflect the changes.

While our local setup instructs `prism-node` to use a local database as the VDR for testing and development, it lacks the benefits of Cardano's secure and decentralized blockchain for publishing DIDs on-chain. To reach the blockchain, our `prism-node` needs to connect to two components:

1. A full node Cardano wallet to submit transactions to the blockchain
2. Cardano-DB-Sync to read the blockchain through a normalized database interface

When the cloud agent needs to publish, update, or deactivate DIDs, it requests `prism-node` to create and validate a transaction, which is then passed to the `cardano-wallet` for submission to the blockchain. Simultaneously, `prism-node` connects to `cardano-db-sync` to read new blocks, filter DID Prisms published on-chain, and notify the cloud agent when the DID operation reaches a certain number of confirmations.

Production and pre-production installations are similar, with the main difference being that production points to `mainnet` and pre-production to `testnet` for both `cardano-wallet` and `cardano-db-sync`. This is achieved by changing environmental variables passed to the Docker containers.

For a production setup, we recommend additional security measures, including changing default passwords, deactivating unnecessary services, and setting up managed database and secret storage providers with regular backups. While these aspects are beyond the scope of this book, we will provide corresponding notes with our recommendations when appropriate.

Hardware recommendations

The Hyperledger Identus cloud agent alone doesn't require too much hardware, any instance with 2GB-4GB ram will run it for testing purposes. Of course as you scale in usage you will naturally want more ram available to handle higher concurrent loads.

The Cardano wallet and DB sync components on the other hand are going to need a lot more resources due to the `mainnet` requirements, according to the official documentation, to run a full Cardano node you need:

- 200GB of disk space (for the history of blocks)
- 24GB of RAM (for the current UTxO set)

Of course, on a production environment you may want to run your agent, wallet and db-sync on different machines connected through a VPN or SSH tunnel in order to isolate them and improve security, e.g., your Cardano wallet may be only connecting to a Cardano node, but not exposed to the Internet. You may also reuse and share your Cardano node instance for your wallet and db-sync components, reducing your hardware requirements.

There are many ways to setup your infrastructure and at least in the beginning, the Cardano node is the most resource demanding of all components.

Configuration

In order to setup `preprod` we recommend copying the config files from the Identus cloud agent and making your own modifications. This is because we will need to modify the docker compose file and that is currently shared among every other type of install such as `local`, `dev` and `multi`. So, to avoid making modifications that will conflict with those defaults, we recommend copying and merging the config into it's own file.

Preparing base config files

1. Copy `infrastructure/local` config into your `preprod` destination, e.g. standing in the `cloud-agent` root directory:

```
cp -rf infrastructure/local infrastructure/preprod
```

2. Copy `infrastructure/shared/docker-compose.yml` for `preprod`:

```
cp infrastructure/shared/docker-compose.yml infrastructure/shared/docker-compose-preprod.yml
```

3. Modify `infrastructure/preprod/run.sh` to point to the new docker compose, the `diff` between the files should look like this:

```
--- local/run.sh    2024-06-18 13:08:47
+++ preprod/run.sh  2024-09-16 17:03:56
@@ -125,5 +125,5 @@

 PORT=${PORT} NETWORK=${NETWORK} DOCKERHOST=${DOCKERHOST} docker compose \
     -p ${NAME} \
-    -f ${SCRIPT_DIR}/../shared/docker-compose.yml \
+    -f ${SCRIPT_DIR}/../shared/docker-compose-preprod.yml \
     --env-file ${ENV_FILE} ${DEBUG} up ${BACKGROUND} ${WAIT}
```

4. Modify `docker-compse-prepod.yml` to disable `postgres` port mapping and to be able to set `DEV_MODE` trough an environment variable:

```
--- docker-compose.yml  2024-06-18 13:08:47
+++ docker-compose-preprod.yml  2024-09-16 17:41:36
@@ -15,8 +15,8 @@
       - pg_data_db:/var/lib/postgresql/data
       - ./postgres/init-script.sh:/docker-entrypoint-initdb.d/init-script.sh
       - ./postgres/max_conns.sql:/docker-entrypoint-initdb.d/max_conns.sql
-    ports:
-      - "127.0.0.1:${PG_PORT:-5432}:5432"
+    #ports:
+    #  - "127.0.0.1:${PG_PORT:-5432}:5432"
     healthcheck:
       test: ["CMD", "pg_isready", "-U", "postgres", "-d", "agent"]
       interval: 10s
@@ -96,7 +96,7 @@
       VAULT_ADDR: ${VAULT_ADDR:-http://vault-server:8200}
       VAULT_TOKEN: ${VAULT_DEV_ROOT_TOKEN_ID:-root}
       SECRET_STORAGE_BACKEND: postgres
```

```
-       DEV_MODE: true
+       DEV_MODE: ${DEV_MODE:-true}
        DEFAULT_WALLET_ENABLED:
        DEFAULT_WALLET_SEED:
        DEFAULT_WALLET_WEBHOOK_URL:
```

5. Modify `.env` file and add environmental variables for Cardano, please note that `NETWORK` variable conflicts with the prism node network, so we are renaming it to `NODE_CARDANO_NETWORK`, your `.env` should look like this:

```
### IDENTUS
AGENT_VERSION=1.33.0
PRISM_NODE_VERSION=2.2.1
VAULT_DEV_ROOT_TOKEN_ID=root

### CARDANO
NODE_CARDANO_NETWORK=preprod
NODE_DB=$PWD/cardano/node-db
WALLET_DB=$PWD/cardano/wallet-db
NODE_CONFIGS=$PWD/cardano/configs
NODE_SOCKET_NAME=node.socket
NODE_SOCKET_DIR=$PWD/cardano/ipc
NODE_TAG=9.1.1
WALLET_TAG=2024.9.3
WALLET_PORT=8090
WALLET_UI_PORT=8091
```

6. Update your `docker-composer-preprod.yml` to add the Cardano wallet service.

```yaml
---
version: "3.8"

services:
  ##########################
  # Database
  ##########################
  db:
    image: postgres:13
    environment:
      POSTGRES_MULTIPLE_DATABASES: "pollux,connect,agent,node_db"
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - pg_data_db:/var/lib/postgresql/data
      - ./postgres/init-script.sh:/docker-entrypoint-initdb.d/init-script.sh
      - ./postgres/max_conns.sql:/docker-entrypoint-initdb.d/max_conns.sql
    #ports:
    #  - "127.0.0.1:${PG_PORT:-5432}:5432"
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres", "-d", "agent"]
      interval: 10s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL:-pgadmin4@pgadmin.org}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD:-admin}
      PGADMIN_CONFIG_SERVER_MODE: "False"
    volumes:
      - pgadmin:/var/lib/pgadmin
```

```
  ports:
    - "127.0.0.1:${PGADMIN_PORT:-5050}:80"
  depends_on:
    db:
      condition: service_healthy
  profiles:
    - debug

##########################
# Services
##########################

prism-node:
  image: ghcr.io/input-output-hk/prism-node:${PRISM_NODE_VERSION}
  environment:
    NODE_PSQL_HOST: db:5432
    NODE_REFRESH_AND_SUBMIT_PERIOD:
    NODE_MOVE_SCHEDULED_TO_PENDING_PERIOD:
    NODE_WALLET_MAX_TPS:
  depends_on:
    db:
      condition: service_healthy

vault-server:
  image: hashicorp/vault:latest
  #    ports:
  #      - "8200:8200"
  environment:
    VAULT_ADDR: "http://0.0.0.0:8200"
    VAULT_DEV_ROOT_TOKEN_ID: ${VAULT_DEV_ROOT_TOKEN_ID}
  command: server -dev -dev-root-token-id=${VAULT_DEV_ROOT_TOKEN_ID}
  cap_add:
    - IPC_LOCK
```

```
    healthcheck:
      test: ["CMD", "vault", "status"]
      interval: 10s
      timeout: 5s
      retries: 5

  cloud-agent:
    image: ghcr.io/hyperledger/identus-cloud-agent:${AGENT_VERSION}
    environment:
      POLLUX_DB_HOST: db
      POLLUX_DB_PORT: 5432
      POLLUX_DB_NAME: pollux
      POLLUX_DB_USER: postgres
      POLLUX_DB_PASSWORD: postgres
      CONNECT_DB_HOST: db
      CONNECT_DB_PORT: 5432
      CONNECT_DB_NAME: connect
      CONNECT_DB_USER: postgres
      CONNECT_DB_PASSWORD: postgres
      AGENT_DB_HOST: db
      AGENT_DB_PORT: 5432
      AGENT_DB_NAME: agent
      AGENT_DB_USER: postgres
      AGENT_DB_PASSWORD: postgres
      POLLUX_STATUS_LIST_REGISTRY_PUBLIC_URL: http://${DOCKERHOST}:${PORT}/cl
      DIDCOMM_SERVICE_URL: http://${DOCKERHOST}:${PORT}/didcomm
      REST_SERVICE_URL: http://${DOCKERHOST}:${PORT}/cloud-agent
      PRISM_NODE_HOST: prism-node
      PRISM_NODE_PORT: 50053
      VAULT_ADDR: ${VAULT_ADDR:-http://vault-server:8200}
      VAULT_TOKEN: ${VAULT_DEV_ROOT_TOKEN_ID:-root}
      SECRET_STORAGE_BACKEND: postgres
      DEV_MODE: ${DEV_MODE:-true}
```

```
      DEFAULT_WALLET_ENABLED:
      DEFAULT_WALLET_SEED:
      DEFAULT_WALLET_WEBHOOK_URL:
      DEFAULT_WALLET_WEBHOOK_API_KEY:
      DEFAULT_WALLET_AUTH_API_KEY:
      GLOBAL_WEBHOOK_URL:
      GLOBAL_WEBHOOK_API_KEY:
      WEBHOOK_PARALLELISM:
      ADMIN_TOKEN:
      API_KEY_SALT:
      API_KEY_ENABLED:
      API_KEY_AUTHENTICATE_AS_DEFAULT_USER:
      API_KEY_AUTO_PROVISIONING:
    depends_on:
      db:
        condition: service_healthy
      prism-node:
        condition: service_started
      vault-server:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://cloud-agent:8085/_system/health"]
      interval: 30s
      timeout: 10s
      retries: 5
    extra_hosts:
      - "host.docker.internal:host-gateway"

  swagger-ui:
    image: swaggerapi/swagger-ui:v5.1.0
    environment:
      - 'URLS=[
        { name: "Cloud Agent", url: "/docs/cloud-agent/api/docs.yaml" }
```

```
      ]'

  apisix:
    image: apache/apisix:2.15.0-alpine
    volumes:
      - ./apisix/conf/apisix.yaml:/usr/local/apisix/conf/apisix.yaml:ro
      - ./apisix/conf/config.yaml:/usr/local/apisix/conf/config.yaml:ro
    ports:
      - "${PORT}:9080/tcp"
    depends_on:
      - cloud-agent
      - swagger-ui

  ##########################
  # Cardano
  ##########################

  cardano-node:
    image: cardanofoundation/cardano-wallet:${WALLET_TAG}
    environment:
      CARDANO_NODE_SOCKET_PATH: /ipc/${NODE_SOCKET_NAME}
    volumes:
      - ${NODE_DB}:/data
      - ${NODE_SOCKET_DIR}:/ipc
      - ${NODE_CONFIGS}:/configs
    restart: on-failure
    #user: ${USER_ID}:${GROUP_ID}
    logging:
      driver: "json-file"
      options:
        compress: "true"
        max-file: "10"
        max-size: "50m"
```

```
    entrypoint: []
    command: >
      cardano-node run --topology /configs/cardano/${NODE_CARDANO_NETWORK}/topology.json
        --database-path /data
        --socket-path /ipc/node.socket
        --config /configs/cardano/${NODE_CARDANO_NETWORK}/config.json
        +RTS -N -A16m -qg -qb -RTS

  cardano-wallet:
    image: cardanofoundation/cardano-wallet:${WALLET_TAG}
    volumes:
      - ${WALLET_DB}:/wallet-db
      - ${NODE_SOCKET_DIR}:/ipc
      - ${NODE_CONFIGS}:/configs
    ports:
      - 127.0.0.1:${WALLET_PORT}:8090
      - 127.0.0.1:${WALLET_UI_PORT}:8091
    environment:
      NETWORK: ${NODE_CARDANO_NETWORK}
    entrypoint: []
    command: >
      cardano-wallet serve
        --node-socket /ipc/${NODE_SOCKET_NAME}
        --database /wallet-db
        --listen-address 0.0.0.0
        --testnet /configs/cardano/${NODE_CARDANO_NETWORK}/byron-genesis.json

    #user: ${USER_ID}:${GROUP_ID}
    restart: on-failure
    logging:
      driver: "json-file"
      options:
        compress: "true"
```

```
        max-file: "10"
        max-size: "50m"
  icarus:
    image: piotrstachyra/icarus:v2023-04-14
    ports:
      - 127.0.0.1:4444:4444
    network_mode: "host"
    restart: on-failure

volumes:
  pg_data_db:
  pgadmin:
  node-ipc:
# Temporary commit network setting due to e2e CI bug
# to be enabled later after debugging
#networks:
#  default:
#    name: ${NETWORK}
```

Cardano wallet

In order to publish DIDs into the VDR, we need to setup our Cardano wallet, the following steps will incorporate the Cardano wallet docker config into our Identus docker config. We will merge whats on https://github.com/cardano-foundation/cardano-wallet/tree/master/run/preprod/docker into our cloud agent setup.

1. Create `snapshot.sh`

```
#! /usr/bin/env -S nix shell 'nixpkgs#curl' 'nixpkgs#lz4' 'nixpkgs#gnutar' --
# shellcheck shell=bash
```

```
set -euo pipefail

# shellcheck disable=SC1091
source .env

# Define a local db if NODE_DB is not set
if [[ -z "${NODE_DB-}" ]]; then
    LOCAL_NODE_DB=./databases/node-db
    mkdir -p $LOCAL_NODE_DB
    NODE_DB=$LOCAL_NODE_DB
fi

# Clean the db directory
rm -rf "${NODE_DB:?}"/*

echo "Network: $NETWORK"

case "$NETWORK" in
    preprod)
        SNAPSHOT_NAME=$(curl -s https://downloads.csnapshots.io/testnet/testnet-db-snapshot.
        echo "Snapshot name: $SNAPSHOT_NAME"
        SNAPSHOT_URL="https://downloads.csnapshots.io/testnet/$SNAPSHOT_NAME"
        ;;
    mainnet)
        SNAPSHOT_NAME=$(curl -s https://downloads.csnapshots.io/mainnet/mainnet-db-snapshot.
        echo "Snapshot name: $SNAPSHOT_NAME"
        SNAPSHOT_URL="https://downloads.csnapshots.io/mainnet/$SNAPSHOT_NAME"
        ;;
    *)
        echo "Error: Invalid network $NETWORK"
        exit 1
        ;;
esac
```

```
echo "Downloading the snapshot..."

if [ -n "${LINK_TEST:-}" ]; then
    echo "Link test enabled"
    echo "Snapshot URL: $SNAPSHOT_URL"
    curl -f -LI "$SNAPSHOT_URL" > /dev/null
    curl -r 0-1000000 -SL "$SNAPSHOT_URL" > /dev/null
    exit 0
fi

curl -SL "$SNAPSHOT_URL" | lz4 -c -d - | tar -x -C "$NODE_DB"

mv -f "$NODE_DB"/db/* "$NODE_DB"/
rm -rf "$NODE_DB"/db

echo "Snapshot downloaded and extracted to $NODE_DB"
```

2. Run `snapshot.sh` in order to sync the required ledger snapshot (you will need `curl`, `lz4` and `tar` installed in your system):

```
bash snapshot.sh
Network: preprod
Snapshot name: testnet-db-70689600.tar.lz4
Downloading the snapshot...
...
Snapshot downloaded and extracted to ./cardano/node-db
```

3. Copy Cardano wallet configs into `./cardano/configs`
4. Run `./refresh.sh` to download the configs.
5. Confirm configs directory for `preprod` look like this:

```
ls cardano/configs/cardano/preprod
alonzo-genesis.json  byron-genesis.json   config.json          conway-genesis.json  download
```

In this chapter we will discuss how to prepare a production envioronment for an Identus application.

We will discuss:

- Hardware recommendations
- Production configuration and security
- Lock down Docker / Postgres (default password hole, etc)
- SSL
- Multi Tenancy
- Keycloak
- Testnet / Preprod env
- Connecting to Mainnet
- Set up Cardano Wallet
- Key Management with HashiCorp
- Connecting to Cardano
- Running dbSync

Maintenance

Mastering Identus means maintaing your application once it's launched.

In this chapter we will cover:

- Restarting and cleanup
- Observability
  - Manage nodes / Memory
  - Performance Testing
  - Analytics with BlockTrust Analytics
- Upgrading Agents
  - How to minimize downtime
- Hashicorp
  - Key Management
  - Key rotation

Section V - Addendum

Trust Registries

Overview of the concept of a Trust Registry

- What role they play in SSI
- Trust Over IP Trust Registry Spec
- Highlight any real world examples if they exist by book completion

Continuing on Your Journey

Information about becoming a Contributor to Identus

- How to contribute to the source code or documentation

Information about how to get involved in the SSI community outside of Identus

Trust over IP (ToIP)

Trust Over IP (ToIP) comprises over 300 organizations from diverse industries collaborating to advance Decentralized Identity through ideas and software. According to ToIP, "We develop tools and specifications to help communities of any size use digital networks to build and strengthen trust between participants."

The ToIP Stack, published in 2019, describes how the four layers (DIDs, DIDComm Protocol, Data Exchange Protocols, and Application Ecosystems) form a secure, scalable, and interoperable digital trust framework. Introduction to ToIP, Version 2.0, released in 2021, is an excellent resource for anyone catching up on the Digital Identity problem and solution. Membership options include free Contributor accounts and paid memberships, providing access to ToIP's resources and Slack channels for collaboration.

To join Trust Over IP, you need to create a free Linux Foundation account and then complete your ToIP application here.

All members agree to open, non-competitive participation, so please have your legal team review all associated documents during the onboarding process.

For further details, refer to The ToIP Stack and Overview.

Decentralized Identity Foundation (DIF)

Appendices

Errata

Errata goes here

We will list any bugs that may have been "printed" in certain editions of the book.

Glossary

Glossary or Index here

This will reference key concepts and Identus terms and where they are mentioned in the book, allowing someone to look up a term and know what section it is referenced in.

TODO: First draft will just lay down the glossary terms, we want to add detailed descriptions later.

VDR Verifiable Data Registry